
A Project Manager's Guide to Agentic-Developed Products

A practitioner's playbook for technical PMs running an AI-agent dev team

Drafted by Claude as a meta-deliverable;
methodology by John Gruber via the roksbnkctl
project

Contents

A Project Manager's Guide to Agentic-Developed Products	4
About this book	4
Acknowledgements & attribution	4
Table of Contents	5
Preface — Who this book is for	5
Part I — The Product Layer	6
Chapter 1. From idea to PRFAQ	6
Why start with a PRFAQ	6
PRFAQ structure	7
Worked example — the roksbnkctl PRFAQ	7
Common PRFAQ mistakes	9
Chapter 2. From PRFAQ to product-level initiatives	9
What an initiative is	9
Sizing rules of thumb	10
Initiatives produce milestones, not features	10
Chapter 3. PRDs that an LLM can actually act on	11
Why most PRDs fail when handed to an agent	11
What a working PRD contains	11
A representative PRD section, from this project	13
What an agent-actionable PRD does not contain	13
Chapter 4. The PRD index — sequencing your initiatives	13
Part II — The Agent Team	14
Chapter 5. The four roles	14
The architect	14
The staff engineer	15
The validator	15
The tech writer	15
Why these four and not others	16
Wave 1 vs Wave 2	16
Chapter 6. The integrator role (you)	16
What the integrator does	17
What the integrator does not do	17
Time budget	18
The integrator's superpower: pattern reuse across sprints	18
Chapter 7. Sprint structure	19
Sprint anatomy	19

Sprint vs PRD: the relationship	19
Sprint 0 is special	20
The release-tag mapping	20
Chapter 8. Prompt engineering, role by role	21
Common structure across all roles	21
Architect prompts	22
Staff engineer prompts	22
Validator prompts	23
Tech writer prompts	24
A note on prompt length	25
Part III — Execution	25
Chapter 9. Bootstrapping Sprint 0	25
What Sprint 0 produces	25
Sprint 0's agent prompts	26
How long does Sprint 0 take?	26
The Sprint 0 → Sprint 1 transition	27
Chapter 10. Running a sprint end to end	27
Day 0 — Sprint 1 closes; Sprint 2 planning begins	27
Day 1 — Drafting the four Sprint 2 prompts	27
Day 1 (afternoon) — Wave 1 dispatch	28
Day 1 (evening) — Wave 1 integration	28
Day 2 — Wave 2 dispatch (tech writer)	29
Day 2 (afternoon) — Tag the release	29
Day 3 onwards — Sprint 3 planning begins	30
Chapter 11. The issues / resolved ledger	30
File naming	30
Issue file format	30
When an issue file should say “no issues filed”	31
The two resolution patterns	31
Severity calibration	31
Reading the resolved files for sprint planning	32
Chapter 12. Watching progress and reporting up	32
The live-watch case: sprintwatch	32
The report-up case: stakeholder updates	33
Anti-patterns for stakeholder reporting	33
Part IV — Patterns & failure modes	34
Chapter 13. When agents disagree	34
API drift between staff and validator	34
Architect vs staff engineer on documentation	34
Validator finds a real bug in staff's code	35
Two agents both edit a shared file	35
Chapter 14. Failure modes and their fixes	35
Half-finished implementations	36

Hallucinated function signatures	36
File ownership conflicts	36
Token-budget exhaustion mid-task	36
Drift between PRD and implementation	37
Integrator burnout	37
Agents commit anyway despite the “do not commit” instruction	37
Tech writer manufactures issues	38
Chapter 15. Patterns and antipatterns	38
DO	38
DON'T	38
Part V — Shipping	39
Chapter 16. From sprint to release tag	39
Release-gate criteria, in order	39
Definition of done — per release	40
Deferred work, captured	40
Release artifacts	40
Chapter 17. Scaling the methodology	41
Two sprints in parallel	41
Two products in parallel	41
Compliance / regulated environments	42
Appendices	43
Appendix A. Real prompt templates (verbatim from roksbnctl)	43
A.1 Architect prompt template	43
Verification before reporting done	44
Final report (under 200 words)	45
Verification before reporting done	46
Issue tracking	46
Final report (under 200 words)	46
A.4 Tech writer prompt template	48
Verification before reporting done	50
Final report (under 200 words)	50
FAQ	54
Acceptance criteria	56
Dependencies	56
Risks	56
Open questions	56
Decided (binding for downstream PRDs)	56
Appendix G. Integrator's pre-flight checklist	57
References	58
Primary sources (the case study)	58
Methodology references	58
Tooling references	59
LLM platform references	59

Index	59
-----------------	----

A Project Manager's Guide to Agentic-Developed Products

A practitioner's playbook for technical product managers running an AI-agent development team from PRFAQ through v1.0.

About this book

This is a working manual for a technical product manager (TPM) who wants to treat large-language-model (LLM) agents as their entire engineering team — architect, staff engineer, validator, technical writer — and ship a real product end to end.

It is not theoretical. Every pattern in this book has been used to ship `roksbnctl`, a multi-thousand-line Go CLI for deploying BIG-IP Next for Kubernetes on IBM Cloud OpenShift, across seven sprints with no human engineers writing code. The agent prompts, the issue ledgers, the sprint definitions, and the resolution logs that drove that work are all preserved in the project's git history and are reproduced here as the running case study.

The book targets a TPM who is comfortable reading code but does not write it for a living, who has shipped products before, and who wants a concrete, copy-pasteable methodology — not an evangelism piece — for using agentic teams in production.

Acknowledgements & attribution

The methodology distilled here was developed during the construction of `roksbnctl` by John Gruber. The agent prompts, sprint plans, PRDs, and issue ledgers reproduced as examples in this book are taken verbatim from that repository's `prompts/`, `docs/`, and `issues/` directories. Their structure is licensed under the same MIT terms as the parent project.

The book itself was drafted by an LLM (Claude) as a meta-deliverable of the same agentic methodology it describes — the technical PM (the integrator) requested the book, gave the structural brief, and the agent produced the manuscript. Errors and overgeneralizations are the agent's; the underlying lived experience is the integrator's.

The four-role pattern (architect / staff / validator / tech writer) borrows liberally from real engineering team structures. The PRFAQ / “working backwards” framing is Amazon's. The sprint cadence is conventional Scrum-adjacent. None of these are original; what is original is wiring them together into a fully-LLM-executed pipeline with a human integrator role.

Table of Contents

Preface — Who this book is for

Part I — The Product Layer - Chapter 1. From idea to PRFAQ - Chapter 2. From PRFAQ to product-level initiatives - Chapter 3. PRDs that an LLM can actually act on - Chapter 4. The PRD index — sequencing your initiatives

Part II — The Agent Team - Chapter 5. The four roles - Chapter 6. The integrator role (you) - Chapter 7. Sprint structure - Chapter 8. Prompt engineering, role by role

Part III — Execution - Chapter 9. Bootstrapping Sprint 0 - Chapter 10. Running a sprint end to end - Chapter 11. The issues / resolved ledger - Chapter 12. Watching progress and reporting up

Part IV — Patterns & failure modes - Chapter 13. When agents disagree - Chapter 14. Failure modes and their fixes - Chapter 15. Patterns and antipatterns

Part V — Shipping - Chapter 16. From sprint to release tag - Chapter 17. Scaling the methodology

Appendices - Appendix A. Real prompt templates (verbatim from roksbnkctl) - Appendix B. File layout reference - Appendix C. Definitions of done, per role - Appendix D. PRFAQ template - Appendix E. PRD template - Appendix F. Sprint plan template - Appendix G. Integrator's pre-flight checklist

References

Index

Preface — Who this book is for

You're a technical product manager. You have shipped one or more software products before. You read code well enough to skim a pull request and tell whether it's roughly doing what was asked. You do not write production code as part of your job — but if a senior engineer were sitting next to you and explained what they were doing, you'd follow along, ask intelligent questions, and catch obvious wrong turns.

You have access to one or more capable LLM agent platforms — Claude Code, Cursor, Aider, custom harnesses — and have used them for one-off tasks ("write me a script that does X"). You have not used them as a team.

You believe, with some hedging, that the next generation of small-to-medium product surfaces — internal tools, developer-facing CLIs, niche SaaS products, technical content

systems — can be built almost entirely by AI agents under human direction, and you want to find out concretely how.

This book is for you. It is **not** for:

- Engineering managers who want to delegate to agents without taking on the integrator role themselves. The integrator role is irreducible; offloading it produces low-quality output. You can pair an EM with an integrator, but somebody is doing the work in this book.
- Founders looking for “build me an MVP overnight” stories. The methodology takes weeks to months for a real product (the case study took ~14 weeks of evening/weekend integrator time across seven sprints). It is not faster than a strong human team. It is cheaper, more consistent in scope, and better documented than most human teams — but not faster on the calendar.
- Developers who want a coding-assistant tutorial. This is about running a team, not about writing better with autocomplete.

If you fit the audience: read Part I and Part II linearly. The remaining parts are reference material — flip to them when you hit the corresponding situation in your own project.

Part I — The Product Layer

Chapter 1. From idea to PRFAQ

Why start with a PRFAQ

A **PRFAQ** is a one-to-three-page document with two sections: a (mock) press release announcing the product as if it shipped, and an FAQ answering the questions a customer, an engineer, and an executive would each ask. Amazon popularized this format under the “working backwards” banner; it works because writing the launch announcement first forces the writer to confront whether the product, as imagined, is actually compelling and concrete.

For an agentic project, the PRFAQ does double duty: it is your check on yourself (is this product real, or am I shaving a yak?), **and** it is the seed corn for every downstream artifact. The PRFAQ's audience benefits become the PRD's “Goal” sections; the PRFAQ's FAQ entries about scope become the PRDs' “Out of scope” lists; the PRFAQ's success metrics become the release-gate definitions of done.

Skip the PRFAQ and your agents have no anchor. They can write anything; without a top-level “this is the product, this is the user,” they will produce locally-coherent code that is globally adrift.

PRFAQ structure

A working PRFAQ has six pieces. The first three are the press release; the remaining three are the FAQ.

1. **Headline + sub-headline.** What did you ship, who is it for, what does it let them do that they couldn't before. One sentence each. If you cannot reduce the headline to one sentence, the product isn't focused enough yet.
2. **Customer problem paragraph.** Three to five sentences. What was the user trying to do? What was in their way? Why now?
3. **Solution paragraph + example use.** Three to five sentences. How does the product solve the problem? Include a concrete example: the actual command they would type, the actual screen they would see.
4. **FAQ — customer questions.** "How do I install it?" "What does it cost?" "Does it work with my existing X?" Five to ten Q&As.
5. **FAQ — internal engineering questions.** "What does this require us to build?" "What's the riskiest dependency?" "What's the smallest version we could ship?"
6. **FAQ — internal executive questions.** "What does success look like?" "What are we not doing?" "How long?"

Length: 800–1,500 words. Less is fine. More is a sign that the product isn't yet sharp.

Worked example — the roksbnctl PRFAQ

To make this tangible, here is a reconstructed PRFAQ for the case-study project (the actual project skipped the PRFAQ step and went directly to PRDs — a mistake the project's own PRD overview flags as "open meta-questions"; this is what the PRFAQ would have looked like, written backwards from what shipped).

PRESS RELEASE — DRAFT, NOT FOR DISTRIBUTION

roksbnctl 1.0 ships: a single binary for deploying BIG-IP Next for Kubernetes on IBM Cloud OpenShift, with no host-tool prerequisites except Terraform.

F5 platform engineers can now roksbnctl up their way to a fully-deployed BNK cluster on ROKS, run end-to-end tests, and tear it back down — without installing kubect1, oc, iperf3, or ibmcloud on their laptop.

The problem. Today, deploying a BIG-IP Next for Kubernetes (BNK) trial on IBM Cloud's Red Hat OpenShift Service (ROKS) takes a Terraform module, four CLI tools (kubect1, oc, iperf3, ibmcloud), a kubeconfig wrangling step, and an SSH jumphost for any pre-cluster setup. The first half-hour of every new deployment goes to environment setup, not to actually deploying or testing BNK. End-to-end tests skip the

throughput phase whenever a developer's machine doesn't have `iperf3` installed.

The solution. `roksbnctl` is a single Go binary that wraps the deployment lifecycle: `init`, `up`, `test`, `down`. It bundles its own `kubectl`-equivalent (via `client-go`), runs `iperf3` in-cluster as a `Job`, and routes pre-cluster operations through a TF-provisioned jumphost over its embedded SSH client. Where the tool needs to delegate (`terraform apply`, occasional `kubectl rollout`), it shells out gracefully; where it can do the work itself, it does. The only host prerequisite is `terraform`.

Example. From a fresh laptop:

```
$ roksbnctl doctor
✓ terraform 1.5.7
✓ workspace bnk-canada (current)
$ roksbnctl up
... (40 minutes of cluster + BNK provisioning) ...
$ roksbnctl test throughput
Uplink: 9.4 Gbps Downlink: 9.2 Gbps
$ roksbnctl down
```

No `kubectl`, no `iperf3`, no `oc`, no `ibmcloud` ever installed. `roksbnctl` handles them.

FAQ — for users

Q: What about my muscle memory? `roksbnctl k get pods`, `apply`, `logs`, `exec`, `port-forward` mirror `kubectl`'s surface byte-for-byte for the resources BNK actually uses. `roksbnctl kubectl <args>` passthrough still works if you prefer.

Q: What if I'm behind a firewall? `roksbnctl --on jumphost <command>` routes execution through your TF-provisioned bastion. No additional config needed; the jumphost is auto-discovered from Terraform outputs.

Q: How do I install it? Single binary; download from GitHub releases, `roksbnctl install` copies it to `~/.local/bin`.

FAQ — for engineering

Q: What's the riskiest piece? The `kubectl`-internalization phase. `client-go`'s API surface churns across versions; pinning to `v0.30.x` and avoiding bleeding-edge features is the mitigation.

Q: What's the smallest shippable version? `v0.7` — `--on jumphost` flag and SSH client only. Solves the firewall pain point without internalizing `kubectl`. Useful as a `v1.0` stepping stone.

FAQ — for executives

Q: How long? Fourteen weeks of engineering time, broken into seven 2-week sprints. Doubling that for real-world overhead targets seven months calendar.

Q: What are we not doing? Multi-cloud (still IBM-only), HCP Terraform integration, Windows full TTY support, a web UI. Each is on the post-1.0 list with rationale.

Q: How will we measure success? The “fresh dev box” test: a developer with only terraform installed can complete the install → up → test → down lifecycle without installing anything else.

That fits on three pages, takes maybe an hour to draft, and constrains the next four months of work. Note what it does not contain: implementation details, code snippets beyond the user-facing example, or a sprint plan. Those come later.

Common PRFAQ mistakes

- **Marketing copy instead of technical commitments.** “Empowers cloud-native teams to unlock seamless workflows” tells your agents nothing. “Replaces four CLI installs with one binary” gives them a concrete target.
- **No explicit non-goals.** The agent will scope-creep happily if you don’t explicitly fence off what’s not being built. The “What we’re not doing” FAQ entry is load-bearing.
- **Skipping the example use.** If you cannot write the actual command line a user will type, you do not yet know what the product is.
- **Overlong.** A four-page PRFAQ is a sign the product spans two products. Split it.

Chapter 2. From PRFAQ to product-level initiatives

What an initiative is

An **initiative** is a discrete, shippable hunk of customer-facing capability that takes one to three sprints to deliver. It is bigger than a feature (“add a `--verbose` flag”), smaller than a product (“CLI for K8s deployments”), and is the unit at which you’ll write PRDs, draft sprint plans, and report progress upward.

The art of breaking a PRFAQ into initiatives is sequencing: each initiative should leave the product in a shippable, demoable state, even if many features are still missing. This both protects you against losing momentum and gives you natural release-tag stopping points.

For `roksbnctl`, the PRFAQ implied roughly six initiatives, sequenced as follows:

#	Initiative	Why this comes here
1	SSH client + --on flag	Smallest concrete value; unblocks initiative 4 (SSH backend)
2	Kubectl internalization	Biggest UX win; eliminates the most-flagged warning
3	Credentials abstraction (cross-cutting)	Informs initiative 4's interfaces; cheap to land before 4
4	Execution backends (local / docker / k8s / ssh)	Largest engineering work; depends on 1, 2, 3
5	DNS probe + GSLB-aware testing	New surface; reuses initiative 4's k8s backend
6	E2E test plan + ship	Validates everything; gates v1.0

Each maps to a single PRD. Each is roughly one to two sprints. Each leaves behind a tagged release (v0.7, v0.8, v0.9, v1.0) that a real user can install and use.

Sizing rules of thumb

- **One initiative $\leq$ three sprints.** If your initiative needs more, it's a product, not an initiative; split it.
- **One initiative $\geq$ one sprint.** If your "initiative" can be done in three days by one agent, it's a feature; bundle it with the surrounding initiative.
- **Each initiative leaves a shippable build.** No initiative should produce code that's only useful when initiative N+1 lands. Cross-cutting work (like the credentials abstraction in the case study) is the exception that proves the rule — and you call it out as cross-cutting in your PRD index.
- **Sequence by dependency, not by appeal.** The most exciting initiative ("execution backends!") is rarely the first one to ship. The first one is whichever unblocks the most downstream work for the lowest cost.

Initiatives produce milestones, not features

Resist the urge to break each initiative into a flat list of features in your tracking system. Each initiative becomes a PRD (next chapter). Each PRD becomes one or more sprints. Each sprint dispatches four agent roles. The unit of feature-level granularity is the task inside an agent's prompt — far below the initiative level.

This separation matters because it lets you talk to executives about initiatives (calendar, customer impact) and to agents about tasks (file paths, acceptance criteria), without conflating the two.

Chapter 3. PRDs that an LLM can actually act on

Why most PRDs fail when handed to an agent

A typical product PRD is written for a human engineering team. It describes the what and the why, sketches the how in broad strokes, and trusts the engineering team to fill in the rest. It is verbose in the parts that are easy (motivation, customer empathy) and thin in the parts that are hard (concrete file paths, acceptance criteria, what's out of scope).

An LLM agent reads such a PRD and confidently generates code. The code will be plausible. It will also probably be in the wrong file, use the wrong library, or interface incorrectly with code your other agents are simultaneously writing. The agent does not have the context that a human engineer accumulates over weeks of code review and Slack conversations.

The fix is not to write longer PRDs. It is to write more concrete PRDs.

What a working PRD contains

The following sections, in this order, are what an agent-targeted PRD must contain. Trim the human-targeted sections; expand the agent-targeted ones.

1. **Why** — one or two paragraphs of motivation. Keep this short; it's not what the agent acts on, but it gives the agent enough to make sensible local decisions when the spec is silent.
2. **Goal** — a single paragraph stating the outcome in measurable terms. "Drop the kubectrl install requirement for the happy path" is a goal. "Improve the UX" is not.
3. **In scope** — bulleted list of capabilities being added. Each bullet is a sentence; aim for five to fifteen bullets.
4. **Out of scope** — equally important. Bulleted list of capabilities NOT being added, with one-line rationale each. This is your guardrail against agent scope creep.
5. **Implementation tasks** — numbered, priority-ordered list of concrete deliverables, each naming the target file path. This is the most important section for agentic work. Example from PRD 02 of the case study:

Order	Item	Files
1	internal/k8s/client.go extension — Build- Clientset(kubeconfig), BuildDynamicClient, BuildOpenShiftClient, in-cluster fallback	edit
2	internal/k8s/get.go — typed + dynamic resource fetcher	new
3	internal/cli/k_get.go — cobra wiring; cli-runtime PrintFlags for -o yaml/json/wide/jsonpath	new

Each row tells an agent exactly what to write and where. The priority ordering is load-bearing: when an agent runs out of token budget, it must know which tasks to drop. The convention “stop at a priority boundary; never half-finish a task” is in the agent’s prompt, not the PRD itself, so the PRD just provides the ordering.

6. **Acceptance criteria** — bulleted list of testable conditions for “done”. These become the validator agent’s test list. Example: “roksbnkctl k get nodes -o yaml is byte-equivalent to kubectl get nodes -o yaml, ignoring managedFields/resourceVersion/creationTimestamp.”
7. **Dependencies** — what other initiatives must land first, what external libraries are added, what the new go.mod / package.json lines are.
8. **Risks** — the things that could go wrong, with proposed mitigations. The validator agent reads this and writes test cases against the risks; the integrator reads it and pre-empts the most likely failures during sprint planning.
9. **Open questions** — explicitly listed, with the proposed default. The agent picks the default unless told otherwise; the integrator tracks open questions for follow-up.
10. **The “Decided” table** (optional but recommended) — a short list of binding contractual decisions (the command surface, the configuration schema) that downstream PRDs must honor. The case study uses this for its CLI surface — once decided, it cannot drift.

A good PRD is 1,500–4,000 words. Shorter and the agent has too little to act on. Longer and you are duplicating work that should live in the PLAN.md or a sprint prompt.

A representative PRD section, from this project

Here is the “In scope / Out of scope” section of [PRD 02 — kubectl internalization](#), reduced to the tone you should adopt:

In scope - roksbnctl k get/apply/describe/delete/logs/exec/port-forward against the BNK-relevant resource set - Output formats -o yaml/json/wide/jsonpath/gotemplate/name byte-equivalent to kubectl for unstructured resources - Top-level aliases for get and logs (the highest-frequency verbs) - Doctor downgrade: kubectl/oc rows become informational, not warnings

Out of scope - Full kubectl command coverage — only what BNK ops actually use, plus the high-frequency passthrough verbs - kubectl rollout, kubectl debug, kubectl wait, kubectl auth — passthrough remains - OpenShift CRD typed clients (Routes, Projects, ImageStreams) — Phase 2.1, deferred to Sprint 5

The “Out of scope” list is what kept the agent from writing 200 helper functions for kubectl debug. Without it, the agent will optimize for completeness; with it, the agent stays in the lane you’ve drawn.

What an agent-actionable PRD does not contain

- **Sprint assignments.** That belongs in the PLAN.md (next chapter). One PRD may be split across several sprints; another may share a sprint with parts of a different PRD.
- **Prompt text.** That lives in prompts/sprint<N>/<role>.md. The PRD is reusable; the prompts are sprint-specific.
- **Marketing language.** “Delight users” is not actionable. “Reduce install time from 30 minutes to 0” is.
- **Debate.** Open questions are listed; debate is resolved before the PRD is dispatched. If you find your agents confused, your PRD is leaking debate.

Chapter 4. The PRD index — sequencing your initiatives

Once you have one PRD per initiative, the **PRD index** (also called the roadmap document, also called PRD 00 in our case study) ties them together. Its job is to:

1. Show the dependency graph between PRDs (initiative N reuses initiative M’s interfaces)
2. Recommend an implementation order (which is rarely the same as the dependency order)

3. List cross-cutting PRDs (like a credentials abstraction that informs every other PRD) and call out their non-linear nature
4. Capture the open meta-questions that cross PRD boundaries — naming conventions, hosting locations, whether to ship as separate releases or bundle them

The PRD index is short — 500 to 1,000 words — because most of the real content has been pushed down into the per-initiative PRDs. It is, however, the single most-read document in the project for the first month. Every agent reads the index in addition to its own PRD; every executive who asks “where are we” reads the index before reading the milestone tracker.

The case study's PRD 00 (`docs/prd/00-OVERVIEW.md`) does this in four sections: Why, Goal, Phasing, and Dependency graph. It then closes with three or four open meta-questions. That structure works; copy it.

After the PRD index, you write the **PLAN.md** — the sprint roadmap. PLAN.md takes the PRD set and decomposes it into sprints with calendar estimates, sprint goals, deliverables, gate criteria, and risks. We'll cover PLAN.md in Chapter 7 (Sprint structure); for now, observe that PRD index + PLAN.md is the full handoff package from Product to Engineering. Once those two documents exist, you can leave the product layer and start the agent-team workflow.

Part II — The Agent Team

Chapter 5. The four roles

Agentic engineering teams in this methodology have **four roles**. Every sprint dispatches one agent per role. The roles partition responsibility; together they cover the full surface of a typical engineering deliverable (design + code + tests + docs).

The architect

Owns: design, infrastructure, book/documentation authoring (the book is the user-facing reference manual; see the case study's `book/src/`), GitHub Actions and CI configuration.

Reads: the PRD for the sprint, `docs/PLAN.md`'s sprint section, the previous sprint's chapters and infra files for tone/style.

Writes: `book/src/` (chapter content), `.github/workflows/*.yml` (CI), Makefile (append-only), README book links, occasionally `docs/` for design notes that aren't yet PRD-grade.

Why this role exists: somebody has to write the user-facing chapter while the feature is fresh in the team's mind; somebody has to extend CI to test the new code; somebody has to write the Mermaid diagrams. Folding this into "staff engineer" means the staff engineer is constantly context-switching; folding it into "tech writer" means it lands too late to influence implementation. Hence a separate role.

The staff engineer

Owens: Go (or whatever language) implementation, refactors, unit tests adjacent to production code (table-driven, no live dependencies), code-level testing utilities.

Reads: the PRD for the sprint, surrounding existing code, the previous sprint's prompts for established patterns.

Writes: `internal/...`, `cmd/...`, `scripts/`, `Makefile` (append-only), `CONTRIBUTING.md` (specific sections only).

Why this role exists: this is the bulk of the work. Most of your sprint's token budget will go here.

The validator

Owens: integration tests, golden-file / byte-equivalence tests, CI matrix expansion, security review (credential audit, leak detection), end-to-end test drivers.

Reads: the PRD, existing tests, the staff engineer's PRD-listed file paths (so the validator can write tests against APIs the staff engineer is concurrently building — see Chapter 13 on the API drift problem this can produce).

Writes: `*_test.go` files in the staff engineer's packages (validator uses the same package name to access unexported symbols), `tools/` for test fixtures, `.github/workflows/` for CI, end-to-end driver scripts, sometimes `cspell.json` and similar lint configs.

Why this role exists: test concerns are different from implementation concerns. The validator's mindset is adversarial — they ask "what if the user does this wrong?" — while the staff engineer's mindset is constructive. Separating the roles forces both perspectives.

The tech writer

Owens: read-only review of everything the other three produced. Identifies tone drift, broken cross-references, examples that don't match the as-built code, missing documentation, and inconsistent terminology.

Reads: everything the other three agents wrote in the sprint — chapter prose, code, tests, CI config, e2e patches, issue files.

Writes: only `issues/issue_sprint<N>_tech-writer.md`. The tech writer never edits production files; their output is a list of findings the integrator handles in the next integration commit.

Why this role exists: quality. The other three roles are heads-down on their own deliverables; without a dedicated reviewer, drift between what the architect documented and what the staff engineer implemented goes unnoticed. The tech writer also catches the kind of “this command doesn’t actually work as written” issues that a real-world user would file as a GitHub issue six weeks after release.

Why these four and not others

The pattern of design + implementation + verification + review is a load-bearing four-element decomposition of engineering work. Other potential roles (security specialist, performance engineer, UX designer) can be added when the project warrants — but for the bread-and-butter case of a CLI / library / internal-tool product, the four covered here are sufficient.

A common temptation is to add a “QA” role separate from “validator.” Resist it. The validator’s brief already covers test design; splitting it produces two roles that re-litigate the same coverage decisions. Same for splitting “architect” into “designer” and “infra” — the integration cost exceeds the specialization benefit.

Wave 1 vs Wave 2

The four roles do **not** all run in parallel.

- **Wave 1: architect, staff engineer, validator.** These three run concurrently. They work on disjoint files (the prompts coordinate this — see Chapter 8) and they file independent issue reports. The integrator gathers their output.
- **Wave 2: tech writer.** Runs after the integrator has folded Wave 1 into a single integration commit. Wave 2 reviews the integrated tree, including any fixes the integrator made during integration, not just the raw agent output.

The two-wave pattern matters because a tech writer with nothing to review is useless, and a tech writer reviewing un-integrated agent output is reviewing the wrong tree.

Chapter 6. The integrator role (you)

Of the five roles in this methodology, four are filled by LLM agents. The fifth — the integrator — is irreducibly human (or, if you like, a separate LLM session managed end-to-end by a human). You are the integrator. This chapter describes what you do.

What the integrator does

In rough order of frequency:

1. **Drafts the four sprint prompts** before any agent is dispatched. The prompts are checked into `prompts/sprint<N>/<role>.md` and committed before dispatch, so the dispatch is auditable and reproducible.
2. **Dispatches Wave 1 in parallel.** Three Agent tool calls in a single message, each with the corresponding prompt file's content as the prompt parameter.
3. **Waits and watches** while the agents run. With `sprintwatch` (Chapter 12) or a similar monitor, this is a passive activity — no interaction needed unless an agent gets stuck.
4. **Aggregates Wave 1 output.** Reads each agent's final report. Reads each agent's `issues/issue_sprint<N>_<role>.md`. Looks at `git status` for the changed files. Resolves merge conflicts when two agents have edited an "append-only-shared" file in incompatible ways.
5. **Triages issues.** For every open issue:
 - **Fix it now** — apply the proposed fix, mark resolved, document briefly in the resolved file.
 - **Accept as-is, with rationale** — document in the resolved file why the issue is acceptable.
 - **Defer** — move it to a future sprint's plan and document the deferral in the resolved file.
6. **Runs the verification gates** — `go build ./...`, `go test ./...`, `go vet ./...`, `gofmt -d -l .`, plus any sprint-specific check. If a gate fails, fix the cause; the agents won't go back and re-run their work without a re-dispatch.
7. **Writes the resolved file(s)** — one `resolved_sprint<N>_<role>.md` per role, documenting how every issue was handled.
8. **Commits the integration** — one commit covering the agents' code, the integrator's fixes, and the resolved files.
9. **Dispatches Wave 2 (tech writer)** with the now-integrated tree as input.
10. **Repeats steps 4-8** for the tech writer's review pass.
11. **Tags the release** if the sprint's gate criteria (per `PLAN.md`) are met. Otherwise, captures the gap in `issues/issue_sprint<N>_blockers.md` and threads it into the next sprint.

What the integrator does not do

- **Write production code.** If you're rewriting an agent's implementation from scratch, your prompt was wrong. Re-dispatch with a corrected prompt.
- **Skip the prompt commit.** It is tempting to dispatch first and commit prompts after; do not. The prompt-first commit is your audit trail.

- **Let the agents commit.** All commits go through the integrator. Agents never have commit authority. This rule exists because (a) each commit needs the resolved-file context an agent doesn't have, and (b) you want one commit per sprint integration, not four agents pushing four commits.
- **Skip the resolved file.** "We didn't have any issues" is rare; "we accepted four issues with rationale" is normal. Document the acceptance reasoning, even briefly, so future sprints have the context.

Time budget

For a "real-world" sprint with one full-day-equivalent of integrator time spread across two weeks of calendar:

- ~3 hours drafting and committing the four prompts (most of this is reading the PRD + prior sprint's prompts)
- ~30 min dispatching Wave 1
- ~2 hours of agent runtime, mostly hands-off (you are doing your day job; sprint-watch tells you when to come back)
- ~2 hours integrating Wave 1 (reading reports, triaging issues, fixing things, writing resolved files, committing)
- ~30 min dispatching Wave 2
- ~1 hour Wave 2 runtime
- ~1 hour integrating Wave 2

That's about 9 hours of integrator time per sprint, give or take. Two-week sprints with one full day of integrator time per week is a comfortable cadence; one full day every other week is achievable but tight.

The integrator's superpower: pattern reuse across sprints

By Sprint 3, you should be reusing 80%+ of Sprint 2's prompt scaffolding — the coordination notes, the issue-file format, the verification block, the final-report shape. Only the task-specific sections change sprint to sprint. The case study's `prompts/sprint2/staff.md` is a near-line-for-line evolution of `prompts/sprint1/staff.md`.

The reason to lean into this pattern reuse is not laziness; it's that agents perform better with familiar prompt shapes. Prompts that consistently structure their input get more consistent output. By Sprint 3, every agent in the project has read three prior sprints' prompts (each new sprint's prompt cites the prior one as a template); the methodology bootstraps itself.

Chapter 7. Sprint structure

A **sprint** is the atomic unit of execution. It corresponds to one PRD-section-worth of work, has a fixed calendar duration (two weeks in the case study), and ends with either a release tag or a gate-not-met document moving the missed work into the next sprint.

Sprint anatomy

Every sprint, as defined in docs/PLAN.md, has exactly seven sections:

1. **Goal** — a single sentence stating what shipping this sprint means. “Ship M2 (v0.8): kubectl no longer required on host for the happy path.”
2. **Code deliverables** — the staff engineer's task table, ordered by priority, with target file paths. (This is also the input to the staff prompt's “Tasks” section.)
3. **Test deliverables** — the validator's task list. Unit, integration, e2e tiers.
4. **Documentation deliverables** — the architect's chapter list (with target filenames in book/src/) and any infra changes.
5. **Gate to next sprint** — the bulleted, testable list of conditions that mark this sprint as “done.”
6. **Risks** — risks specific to this sprint's work, with mitigations.
7. **Book chapter targets** (if applicable) — which user-facing reference chapters land at the end of this sprint.

That's it. Sections like “Definition of Done” or “Acceptance Criteria” are absorbed into “Gate to next sprint.” Sections like “Stretch Goals” are explicitly avoided (they invite scope creep; if a stretch is interesting, it goes in the next sprint).

Sprint vs PRD: the relationship

A PRD describes a feature initiative end-to-end. A sprint describes two weeks of work. Sometimes a PRD fits in one sprint; sometimes it spans two; occasionally one sprint touches two PRDs (when a small PRD is bundled with the back half of a large one).

PLAN.md is the document that maps PRDs to sprints. The case study's mapping looks like this:

```
Sprint 0 → (foundations; no PRD)
Sprint 1 → PRD 01 (SSH/--on) → tag v0.7
Sprint 2 → PRD 02 (kubectl) → tag v0.8
Sprint 3 → PRD 04 (creds) + PRD 03 (first half: local + docker backends)
Sprint 4 → PRD 03 (second half: k8s + ssh)
Sprint 5 → PRD 03 (DNS) + polish → tag v0.9
Sprint 6 → PRD 05 (E2E)
Sprint 7 → Book launch + polish → tag v1.0
```

Note Sprint 3 deliberately starts with the cross-cutting PRD 04 because it informs PRD 03's interfaces. Note Sprint 7 has no new PRD: it's a polish sprint dedicated to dogfooding and the v1.0 cut.

Sprint 0 is special

Sprint 0 is your **foundations sprint**: CI matrix, pre-commit hooks, doc infrastructure, the issue-file format, the prompt-template structure. It produces no user-facing features; its output is the runway for Sprints 1-N.

The reason Sprint 0 exists is that without it, every subsequent sprint has to spend time on infrastructure-of-infrastructure. With Sprint 0 done, every subsequent sprint is purely feature-and-doc work.

The case study's Sprint 0 produced:

- Expanded CI matrix (Linux + macOS, gofmt + vet + staticcheck)
- A pre-commit hook running unit tests
- The mdBook book/ skeleton with all 32 chapter stubs
- The GitHub Pages publishing workflow for the book
- A doctor-command refactor making per-backend checks possible
- The tools/docker/ Makefile for future tool-image builds

That seems like a lot for a “no features” sprint. It is. Sprint 0 is your most-front-loaded sprint, and it's worth the upfront cost.

The release-tag mapping

Each sprint either tags a release or doesn't. The case study tags every two-to-three sprints:

Sprint	Tag	Why this sprint, this tag
0	(none)	Foundations only; nothing to ship to users
1	v0.7	First user-facing feature (SSH/-on); 6 chapters live; M1 milestone
2	v0.8	KubectI-equivalent shipped; 13 chapters; M2
3	(none)	Cross-cutting + half a PRD; not a user-facing milestone yet
4	(none)	Other half of PRD 03; not yet a usable backend matrix
5	v0.9	All four backends + DNS; M3
6	(none)	E2E test sprint; gates v1.0 but doesn't ship it
7	v1.0	Polish + dogfood + book launch

The lesson: not every sprint ships a tag. Tag when the user has something material that wasn't usable before. Bunching multiple sprints behind a single tag (Sprints 3-5 → v0.9) is fine when the intermediate sprints don't leave the product in a useful state on their own.

Chapter 8. Prompt engineering, role by role

This is the longest chapter in the book and the most concrete. It goes role by role through the structure of an agent prompt as used in production, with annotated excerpts from the case study's prompts/sprint2/*.md files. (The full files are reproduced in Appendix A.)

Common structure across all roles

Every prompt — architect, staff, validator, tech writer — has the same nine-section skeleton:

1. **Role + sprint identification** — one-line opener: “You are the X agent for Sprint N of the roksbnkctl project.” Sets the agent's role.
2. **Project location** — the absolute path to the repo. Agents run with no implicit cwd. “Project location: /mnt/d/project/roksbnkctl/. Go module github.com/jgruberf5/roksbnkctl. Min Go: 1.25.”
3. **“Read first”** — bulleted list of files the agent must read before starting work. The PRD, the relevant docs/PLAN.md section, the previous sprint's analogous prompt, and any code the agent will be modifying. **This section is the single biggest predictor of agent quality.** A prompt with a thorough “Read first” produces aligned work; a prompt without it produces locally-coherent but globally-drifting work.
4. **Coordination notes** — names the other agents running in parallel, their owned files, and which files are append-only-shared. Critical for preventing two agents from clobbering each other.
5. **Numbered tasks** — the heart of the prompt. Concrete deliverables in priority order, each with target file path, brief acceptance criteria, and ideally a code snippet or signature that the agent should produce.
6. **Issue tracking format** — the exact markdown schema for the agent's issues/issue_sprint<N>_<role>.md file. Reproducing it in every prompt avoids drift.
7. **Verification before reporting done** — checklist the agent must pass before declaring complete. “go build ./... clean. go test ./... clean. gofmt -d -l . clean.”

8. **Final report shape** — constrains the agent's user-facing summary. "Under 200 words. Files created. Files edited. Test results. Issues filed. What the integrator should know."
9. **"Do NOT commit"** — explicit instruction that only the integrator commits. Belongs in every prompt; the agent will commit unprompted otherwise.

Sections 1, 2, 6, 7, 8, 9 are nearly identical across roles within a sprint and across sprints — they're your project's house style. Sections 3, 4, 5 are role- and sprint-specific.

Architect prompts

The architect prompt focuses on design + documentation + infrastructure. Its key idiosyncrasies:

- **"Read first" emphasizes prior sprint chapters for tone consistency.** Reference Sprint 1's chapter 16 when writing Sprint 2's chapter 24, so the voice doesn't drift sprint to sprint.
- **Tasks are typically chapter-by-chapter.** "Replace stub at book/src/05-doctor.md" with "Coverage of: doctor command end to end. Sections worth covering: ..." — the prompt enumerates the sub-headers the chapter should have.
- **Style guidance section.** "Lower-case prose; sentence-case section headers. Code blocks for any command; inline code for filenames. Cross-reference other chapters with relative links. PRD links use GitHub-canonical URLs." — these are easy for an agent to forget unless explicitly listed.
- **"Verify against staff engineer's actual implementation"** — when chapter content depends on the staff agent's choices, the architect prompt explicitly tells the architect to read the staff agent's code (which is being written in parallel) and update the chapter to match. This produces some drift the tech writer catches in Wave 2; that's by design.

Excerpt from the case study's prompts/sprint2/architect.md:

```
Chapter 24's example commands appear in the actual `cmd/roksbnkctl --help`
output once staff lands their work – coordinate with staff if there's
drift, or note as an issue for the integrator and tech-writer to
reconcile
```

The prompt explicitly licenses the architect to flag drift as an issue rather than block on it. Combined with the tech writer's Wave 2 review, this catches most drift before release.

Staff engineer prompts

The staff engineer prompt focuses on code + correctness. Its key idiosyncrasies:

- **Priority-ordered tasks with “stop at boundary” license.** Critical:

If you run out of token budget, stop at the priority boundary you reached and file an issue describing what's deferred. Don't half-finish a task.

Without this, an agent at 90% of its budget will rush to half-implement task 8, leaving everything 1-8 in a partially-broken state. With this, the agent finishes task 7, files an issue saying “task 8 deferred to next sprint,” and lands a coherent commit.

- **API signatures inline.** When you know what a function should look like, write the signature in the prompt:

```
func BuildClientset(kubeconfigPath string) (kubernetes.Interface, error)
```

This eliminates ambiguity about parameter naming, return shape, error vs panic, and so on.

- **Coordination with validator.** The staff engineer is told explicitly: “Don’t write *_test.go files in internal/k8s/ — validator owns those.” Without this, both agents will write tests, and the integrator will have to merge them.
- **Verification block emphasizes binary-level smoke tests.** Not just compilation:

```
- roksbnctl --help shows the new k parent + at least the top-level aliases
- roksbnctl k --help shows the verb list
- roksbnctl get --help works (alias)
```

Agents will frequently produce code that compiles but doesn’t wire up to the CLI surface correctly. Binary-level checks catch this.

Validator prompts

The validator prompt focuses on tests + CI + security. Its key idiosyncrasies:

- **The validator writes tests in the staff engineer’s package.** This is unusual but load-bearing: the validator needs to access unexported symbols, and using package k8s (same as staff’s production code) gives them that visibility. The prompt is explicit:

Use the same package name (`package k8s`) so you can access unexported symbols.

This causes the API drift problem covered in Chapter 13. The mitigation is also in the prompt: validator and staff coordinate on the API surface during the sprint.

- **Tiered tests with explicit gating.** Unit tests run on every commit; integration tests need a live dependency (Docker, kind cluster) and run in a separate CI lane;

golden-file tests need a real production-like environment and are gated behind `-tags=live`. The prompt enumerates which tier each test belongs to.

- **Security review responsibilities are explicit.** The case study's PRD 04 (credentials) instructs the validator to audit that API keys never appear in `docker inspect`, in command-line `argv`, in kube events, or in any log file readable by another local user. This is in the validator prompt, not just the PRD, so it gets executed.
- **The validator may file Severity: roadmap issues.** Forward-looking observations that aren't blocking the current sprint but should inform a future sprint go in as roadmap severity. The integrator threads them into the next sprint's planning. From the case study:

Severity: roadmap is reserved for non-blocking forward-looking observations; low/medium/high/blocker for actionable findings.

Tech writer prompts

The tech writer prompt is the simplest and the strangest. It says, in effect, "read everything the others did; file issues; don't edit anything except your own issue file."

- **Read-only.** The prompt is explicit: "Do not edit any files except `issues/issue_sprint<N>_tech-writer.md`."
- **Must run after Wave 1 integrates.** The tech writer reviews integrated code, not raw agent output.
- **Looks for drift.** The prompt gives the tech writer a list of things to look for:
 - PRD vs implementation mismatch (did the staff engineer ship what the PRD specified?)
 - Documentation vs implementation mismatch (do the architect's example commands actually work?)
 - Tone consistency across chapters
 - Cross-reference correctness
 - Sample-output realism
- **No false positives.** The prompt tells the tech writer not to manufacture issues for the sake of finding something:

Don't manufacture issues; clean reviews are valid. If you find nothing, file with the heading and `"*No issues filed.*"`

Sprint 2's architect file in the case study actually does this — `*No issues filed.*` for a clean review pass.

A note on prompt length

The case study's prompts run 100–200 lines (~1,000–2,500 words) each. That's longer than a typical “give me a script” prompt and shorter than a typical “write me a 10-page document” prompt.

Shorter prompts (under 500 words) produce variable output: the agent makes too many local decisions you didn't intend. Longer prompts (over 4,000 words) produce diminishing returns: the agent attends less consistently to the back half. The 1,000–2,500 word range is the sweet spot.

If your prompt is creeping past 2,500 words, suspect that you're trying to put two roles' worth of work into one role. Split the work, not the prompt.

Part III — Execution

Chapter 9. Bootstrapping Sprint 0

You are at week 0. You have a PRFAQ, a PRD index, and a PLAN.md. You have a git repo. You have not yet dispatched an agent.

This chapter walks through Sprint 0 — the foundations sprint — concretely.

What Sprint 0 produces

Sprint 0's deliverables are infrastructural, not feature-bearing. They are:

1. **CI matrix.** GitHub Actions (or equivalent) running unit tests on at least two OSes (Linux is mandatory; macOS strongly recommended; Windows compile-check is fine for Go projects). Add gofmt, go vet, staticcheck (or your language's equivalent linters).
2. **Pre-commit hook.** Local-developer-quality gate: gofmt + vet + fast unit tests. The hook runs in <30 seconds; anything slower belongs in CI.
3. **Issue-file format established.** Create issues/README.md documenting the format you will use for the rest of the project (severity values, status values, file naming convention `issue_sprint<N>_<role>.md`). The case study's issues/README.md is a perfect template.
4. **Prompt directory structure.** prompts/README.md documenting the dispatch playbook, the role definitions, and the kick-off checklist. Again, the case study's prompts/README.md is reusable wholesale.

5. **Documentation infrastructure.** If your product has a user-facing reference manual (and most should), set up the toolchain now: mdBook for Rust-y projects, MkDocs for Python-y, Docusaurus for full-blown JavaScript stacks, etc. Wire the publish workflow to GitHub Pages or wherever. Stub every chapter the PLAN.md eventually expects, even if 90% are placeholder text.
6. **A doctor-equivalent self-check tool.** For a CLI: a command that checks the user's environment and reports green/yellow/red. For a library: an installer that confirms the runtime is set up. Sprint 0 establishes the shape of this; later sprints fill in checks.
7. **The PRD index + PLAN.md committed.** If you haven't yet, do it now. Sprint 1's prompts will reference these files.

Sprint 0's agent prompts

Sprint 0 dispatches the same four roles, but the prompts are unusually short because the deliverables are bounded:

- **Architect:** "Stand up the book infrastructure. Create stubs for all 32 chapters per the PLAN.md outline. Wire the publish workflow. Add a top-of-README book badge."
- **Staff engineer:** "Refactor internal/doctor/ into a Check{Name, Status, Detail} struct that future sprints can extend. Add CI matrix configuration. Write the pre-commit hook."
- **Validator:** "Set up tools/docker/ skeleton + GitHub Actions tool-image-build workflow. Add a spell-check workflow. Write the smoke-test section of CONTRIBUTING.md."
- **Tech writer:** "Review all of the above for consistency, completeness against PLAN.md Sprint 0 deliverables, and reusability into Sprint 1."

The case study's actual Sprint 0 prompts are at prompts/sprint0/*.md and run 60-120 lines each. Sprint 0's prompts are also where you finalize your project's house style — the issue file format, the verification block contents, the final-report shape — so that Sprints 1+ can copy the templates wholesale.

How long does Sprint 0 take?

The case study's Sprint 0 took roughly one calendar week of integrator part-time work (roughly 4-6 integrator-hours) plus 4-8 hours of agent runtime. Everything you do in Sprint 0 saves time later, but the upfront cost is real. Don't compress it; it costs you tenfold in Sprints 1+ if you do.

The Sprint 0 → Sprint 1 transition

When you tag the end of Sprint 0 (no release; just a “Sprint 0 complete” commit), you should be able to:

1. Open `prompts/sprint0/staff.md` and see a complete prompt that, if re-dispatched today, would reproduce the work.
2. Open `issues/resolved_sprint0_*.md` and read the integrator's notes on every issue the agents filed.
3. Run `make book` (or your equivalent) and see a renderable book with all chapter stubs in place.
4. Open `docs/PLAN.md` Sprint 1 section and see a deliverables list specific enough to draft Sprint 1's four prompts in 2-3 hours.

If any of those is missing, finish Sprint 0 before dispatching Sprint 1. The temptation to “we'll fix it as we go” is high; the cost of fixing it during Sprint 1 is roughly 2x the cost of fixing it now.

Chapter 10. Running a sprint end to end

This chapter walks Sprint 2 of the case study end-to-end as a concrete example.

Day 0 — Sprint 1 closes; Sprint 2 planning begins

The integrator has just tagged v0.7 (Sprint 1's release). Before doing anything else, they:

1. Read `docs/prd/02-KUBECTL-INTERNAL.md` end to end.
2. Read `docs/PLAN.md` Sprint 2 section.
3. Read all `issues/issue_sprint1_*.md` and triage which open issues roll into Sprint 2 (most don't; a few do).
4. Read `prompts/sprint1/*.md` — the four Sprint 1 prompts — as templates.

Time spent: ~90 minutes.

Day 1 — Drafting the four Sprint 2 prompts

The integrator writes four prompt files, in this order:

1. `prompts/sprint2/architect.md` — chapter targets (5, 6, 8, 9, 10, 11, 24), each with sub-headers; coordination notes naming the staff and validator agents; verification block requiring `mdbook build` to succeed.

2. **prompts/sprint2/staff.md** — 10 priority-ordered tasks per PRD 02's implementation list; coordination note that validator owns *_test.go files; Go signatures inline for new exports.
3. **prompts/sprint2/validator.md** — fake-clientset unit-test plan; golden-file byte-equivalence test plan; CI updates; e2e Phase D patch with PATH-strip step; CONTRIBUTING.md "Running golden tests" section.
4. **prompts/sprint2/tech-writer.md** — review checklist tuned for Sprint 2's specific concerns (chapter 24's example commands must work against staff's actual implementation).

The prompts overlap zero — a deliberate property checked by the integrator's "pre-flight" pass (Appendix G).

The integrator commits these four files in their own commit:

```
prompts/sprint2: draft 4 agent prompts ahead of dispatch
```

Time spent: ~3 hours.

Day 1 (afternoon) — Wave 1 dispatch

The integrator dispatches three agents in parallel via three Agent tool calls in a single message. Each prompt file's content goes in verbatim as the prompt parameter.

Total agent runtime: ~90 minutes (the slowest agent — staff — does the most work).

The integrator goes back to their day job, glancing at sprintwatch in another terminal occasionally.

Day 1 (evening) — Wave 1 integration

When all three agents return:

1. Integrator runs `git status` — sees ~30 modified or new files across `internal/k8s/`, `internal/cli/`, `book/src/`, `.github/workflows/`, `scripts/`.
2. Reads each agent's final report.
3. Reads each agent's `issues/issue_sprint2_*.md`.

In Sprint 2's case, the agents flagged 14 issues total: 4 closed by the agents themselves, 10 left open. The integrator triages:

- **6 issues** — fix immediately, edit production code, mark resolved with rationale in `resolved_sprint2_<role>.md`.
- **3 issues** — accept as-is, with rationale (e.g., "OpenShift CRDs deferred to Phase 2.1 per PLAN.md").

- **1 issue** — defer to Sprint 5 (kind-based CI integration; PLAN.md sequences this for Sprint 4's k8s backend work, so it makes sense to bundle).

The integrator runs verification: `- go build ./... ✓ - go test ./... ✓ - go vet ./... ✓ - gofmt -d -l . ✓ - roksbnctl --help shows new commands ✓ - mdbook build book/ ✓`

Writes three `resolved_sprint2_<role>.md` files, one per Wave 1 agent.

Commits:

Sprint 2: `kubectl internalisation + 7 v0.8 chapters (PRD 02)`

Time spent: ~2 hours.

Day 2 — Wave 2 dispatch (tech writer)

The integrator dispatches the tech writer agent with `prompts/sprint2/tech-writer.md` as the prompt. The tech writer reads the integrated tree and files `issues/issue_sprint2_tech-writer.md` with whatever it found.

Sprint 2's tech writer filed 10 issues: 6 medium-severity (chapter examples that didn't quite match the staff's API), 3 low-severity (cross-reference fixes), 1 informational. The integrator processes them the same way as Wave 1 — fix, accept, or defer — and commits:

Sprint 2: `tech-writer integration – 10 issues resolved`

Time spent: ~1 hour.

Day 2 (afternoon) — Tag the release

The PLAN.md Sprint 2 gate criteria say:

- M2 merged + tagged v0.8 ✓
- E2E with `kubectl PATH-stripped` passes on a live cluster ✓
- Byte-equivalence test passes for `get -o yaml` on Node, Pod, Service, ConfigMap ✓
- Seven chapters above published; book TOC reflects the new structure ✓

All four met. Integrator runs `git tag v0.8 -m "..."` and `git push origin v0.8`. GitHub Actions builds the release artifacts. v0.8 is live.

Total integrator time across Sprint 2: ~7 hours over 2 days. The agents' total runtime was ~3 hours.

Day 3 onwards — Sprint 3 planning begins

The cycle repeats. The integrator reads PRD 03 and PRD 04, reads Sprint 2's prompts as templates, and starts drafting Sprint 3's four prompts.

Chapter 11. The issues / resolved ledger

The `issues/` directory is the project's working memory across sprints. This chapter formalizes its conventions.

File naming

```
issues/issue_sprint<N>_<role>.md      # filed by the agent during sprint N
issues/resolved_sprint<N>_<role>.md    # written by the integrator post-sprint
```

`<N>` is the sprint number (zero-padded to two digits if you have ≥ 10 sprints, but most projects have < 10). `<role>` is one of `architect`, `staff`, `validator`, `tech-writer`.

Issue file format

Each issue is a markdown section:

```
## Issue N: short title
**Severity**: low | medium | high | blocker | roadmap
**Status**: open | in-progress | resolved | wontfix
**Description**: what was found, what was expected
**Files affected**: list of paths
**Proposed fix**: how to resolve
**Related**: links to PRDs, other issues, commits
```

- **Severity** is the agent's call. `roadmap` is reserved for non-blocking forward-looking observations.
- **Status** at file time is usually `open` (or `resolved` if the agent fixed it themselves before the issue was even filed). The integrator may flip statuses during integration.
- **Description** must include both what was found and what was expected so the integrator can decide whether the agent's expectation was correct.
- **Proposed fix** is how the integrator avoids the "I read the issue, now what?" moment. The agent's proposed fix is rarely wrong; sometimes the integrator merges it, sometimes they substitute a better one.

Issue files may also contain prose between sections — e.g., a Verification status table at the end of the file showing what the agent did/didn't run before reporting. Include whatever the agent has that the integrator will need.

When an issue file should say “no issues filed”

A clean review is a valid outcome. If the agent finds nothing wrong, the file looks like:

```
# Sprint 2 – architect issues
```

```
*No issues filed.*
```

```
[Brief prose summary of what was done, ~100 words.]
```

The “no issues filed” marker is a sentinel — sprintwatch and other tooling rely on it to distinguish “agent ran clean” from “agent never ran.” The brief prose summary is optional but useful for the integrator’s audit trail.

The two resolution patterns

When an issue is fixed, the integrator can:

1. **Edit the issue in place.** Flip the status to resolved, add a Resolution paragraph and commit SHA. This works for small issues.
2. **Write a separate resolved_*.md file.** Per-role companion file with one section per issue, each documenting how the issue was handled. This works for sprints where the resolutions are substantive enough to warrant their own document.

Pattern 1 is fine for issues with one-line fixes. Pattern 2 is preferred for sprints where the integrator made architectural decisions during integration; you want those decisions written down for the next sprint’s planners.

The case study uses pattern 2 consistently. It makes the audit trail cleaner.

Severity calibration

Without calibration, every agent will inflate severities (“this is high!”). Once you’ve run two or three sprints, write down what each severity actually means in your project. Ours, distilled from the case study:

- **blocker** — sprint cannot ship. The release tag is gated on resolving this. Rare; a few per project lifetime.
- **high** — known correctness bug that affects users in expected workflows. Fix this sprint or document why it’s deferred.
- **medium** — incorrectness in a less-trodden path, or a design issue with significant downstream cost. Most issues will end up here.
- **low** — cosmetic, minor quality, or “we’d like this better” — not affecting correctness. Often deferred.
- **roadmap** — forward-looking observation that’s not actionable in this sprint but should inform planning for a future sprint.

Re-run severity calibration with your team (you, plus the next-sprint's prompt drafts) every few sprints. Severity drift is a real thing.

Reading the resolved files for sprint planning

When you draft Sprint N's prompts, you read all `issues/issue_sprint<M>_*.md` for $M < N$ and triage. The resolved files are equally important: they capture the reasoning behind the integrator's prior decisions. Don't re-litigate decisions you've already made; let the resolved file end the debate.

Chapter 12. Watching progress and reporting up

You have agents running. You have an integrator (you). You have a stakeholder (your boss, your customer, your team) who wants to know "where are we." This chapter covers tooling and habits for both the live-watch case (during a sprint) and the report-up case (between sprints).

The live-watch case: sprintwatch

The case study includes a small terminal dashboard, `tools/sprintwatch/`, that reads the `issues/`, `resolved_*.md`, and `prompts/sprint<N>/` directories and shows live sprint progress. It is reproduced (in concept) here as a worked example of "what to build" — your own implementation may differ in detail.

The dashboard shows, per sprint:

- A header with sprint number and overall status (✓ COMPLETE, ► IN PROGRESS, etc.)
- A roles row: ✓ for done, ✗ for open issues, ? for not yet reported
- Issue counts: filed, resolved, deferred, open
- Severity breakdown of open issues (blocker / high / medium / low)
- An ASCII burn-down sparkline reconstructed from `git log` history of the issue files
- Either a Closed: <date> line (for completed sprints) or an ETA: <date> projection (for in-progress sprints, computed from velocity over a trailing window)

The point of building this is not "we needed a dashboard." It's that:

1. Without a live view, the integrator either spams `git status` and `cat issues/*.md` constantly, or loses track of where each agent stands.
2. The dashboard is unintrusive — it reads files; it does not depend on agents emitting structured output. The agents work the same way whether or not anyone is watching.

3. The dashboard's existence drives discipline. If you can see the per-sprint open count, you stop letting open counts grow unchecked.

A working version of sprintwatch's source is in [the case-study repo](#). Forking it for your own project takes about an hour.

The report-up case: stakeholder updates

Stakeholders don't want to read the dashboard. They want a one-paragraph status with:

- **What's done since last update.** Reference the closed sprint's release tag if applicable.
- **What's next.** Name the next sprint's PRD and goal.
- **Any blockers.** Name them, with whether you need a decision from the stakeholder.
- **Forecast.** When does the next milestone tag land. Provide a range (e.g. "v0.9 in 4-6 weeks"), not a single date — agentic work has variance.

For the case study, the cumulative stakeholder update at the end of Sprint 2 would have read:

Sprint 2 closed; v0.8 tagged. The kubectI host-install requirement is dropped for the everyday workflow. Book is at 13 chapters published, on schedule for the 24-chapter target by end of Sprint 5. Sprint 3 starts Monday on the credentials abstraction (PRD 04) and the local + docker execution backends (PRD 03 first half). One open question: should we host backend tool images under `ghcr.io/jgruberf5/...` or under an F5-owned org? — needs a stakeholder decision before Sprint 4. Forecast: v0.9 in 6-8 weeks (Sprints 3-5); v1.0 in 4-5 months.

That's three sentences for what got done, one for what's next, one with the open decision, one with the forecast. About 100 words. Send weekly during a sprint, update post-tag for milestones.

Anti-patterns for stakeholder reporting

- **Velocity charts.** Don't. Agentic velocity is not meaningful at the sprint level; it's meaningful at the release level. Report on tags shipped, not story points completed.
- **Burndown charts in stakeholder updates.** The internal sprintwatch view is for you. The stakeholder doesn't care about issue counts; they care about "is the release on track."
- **Hiding deferred work.** Always name what was deferred and why. Stakeholders catch on quickly that "we're on track" + "we're carrying 14 medium-severity issues from prior sprints" is a contradiction. Be explicit and you keep credibility.

Part IV — Patterns & failure modes

Chapter 13. When agents disagree

Wave 1 agents run in parallel. They cannot directly communicate. They can only coordinate through the prompts the integrator wrote and through the files they each touch.

This means agents will sometimes disagree. The disagreements take a few characteristic shapes.

API drift between staff and validator

The most common conflict. The validator agent writes tests against the API the PRD specifies; the staff engineer writes the actual code, and during implementation may rename a field, change a parameter order, or merge two functions into one. The validator's tests now fail to compile.

The case study saw this in Sprint 2 — concretely, the validator wrote tests calling `Apply(ApplyOptions{Dynamic, Paths, Stdin, ...})` while the staff engineer's final API was `(*ApplyOptions).Run(ctx) with Filename`, not `Paths/Dynamic/Stdin`. The validator's tests didn't compile against the staff's code.

Mitigation: the staff prompt's API signatures section. If you specify the function signatures inline in the prompt, both agents see the same signatures and drift is reduced.

Resolution when drift happens anyway: the integrator chooses one side (usually the staff engineer's, since that's the API that ships) and either rewrites the validator's tests to match, or files an issue documenting the choice and threads the rewrite into the next sprint. Either is fine; the case study's Sprint 2 explicitly chose the staff engineer's surface and asked the integrator to rewrite the validator's tests at integration time.

The case study filed this as Sprint 2 Issue 6 in the staff engineer's issue file ("validator API mismatch — coordination gap") with severity medium, status Δ for the integrator — coordination gap. The integrator picked it up at integration time and resolved it by rewriting the affected validator tests. The pattern: flag the gap, integrate, fix during integration.

Architect vs staff engineer on documentation

The architect documents what the spec says the feature should do. The staff engineer ships what the code actually does. When the staff engineer's choices diverge from the spec — even slightly — the architect's chapter is now subtly wrong.

Mitigation: the architect prompt should explicitly tell the architect to read the staff engineer's actual implementation. From the case study:

Read the staff agent's actual implementation — `internal/cli/k_*.go` files — to verify your example commands work as-written. Don't paraphrase from PRD 02; the staff agent's choices may have diverged.

Resolution when drift happens anyway: the tech writer (Wave 2) catches it. The tech writer's prompt explicitly checks chapter examples against the implementation. Tech writer files an issue; integrator fixes the chapter.

Validator finds a real bug in staff's code

The validator's tests fail because there's a real problem in the staff engineer's implementation. The validator files an issue with severity high or medium. The integrator must decide:

- **Fix the staff code.** The integrator (you) edits the staff agent's production code to fix the bug. Document the fix in `resolved_sprint<N>_validator.md` ("Fixed by integrator: changed line 42 of `internal/k8s/get.go` from X to Y").
- **Re-dispatch the staff agent** with a corrected prompt. Rare; usually only worth it if the bug indicates a systematic issue across many of the staff's deliverables.

The case study saw both. By Sprint 5 the integrator was confident enough in the agent stack to fix bugs at integration time rather than re-dispatch.

Two agents both edit a shared file

Some files are unavoidably shared — `Makefile`, `CONTRIBUTING.md`, `go.mod`. The convention is to mark these as **append-only-shared** in the prompts. Each agent appends; nobody overwrites.

When this fails — two agents both edit the same line — the integrator merges manually. It's annoying but rare.

Mitigation: explicit ownership in prompts. The case study's Sprint 2 staff prompt says: "You own everything else" (after listing what validator owns), which leaves no ambiguity.

Chapter 14. Failure modes and their fixes

This chapter catalogs the failure modes you will encounter and what to do about each.

Half-finished implementations

Symptom: an agent's report says "I implemented tasks 1–7 and started task 8, but it's not quite working." Code is in a broken state.

Root cause: the agent ran out of token budget mid-task, didn't have permission to defer, and rushed to almost-finish.

Fix: the staff engineer prompt's "stop at boundary" instruction (Chapter 8). Re-issue with the boundary instruction explicit:

If you run out of token budget, stop at the priority boundary you reached and file an issue describing what's deferred. Don't half-finish a task.

This is so important that it goes verbatim into every staff engineer prompt.

Hallucinated function signatures

Symptom: code references a function or library API that doesn't exist. Compilation fails.

Root cause: the agent guessed at an API surface it didn't read.

Fix: the "Read first" section of the prompt is your only real defense. Make it thorough. Specifically: if the agent will use library X, name the relevant docs URL or, better, paste the relevant function signatures into the prompt itself.

Some hallucination is inevitable. Plan for the integrator to fix hallucinated calls during integration; budget ~30 minutes per sprint for this.

File ownership conflicts

Symptom: `git status` shows conflicting modifications to the same file by two agents.

Root cause: the prompts didn't clearly partition file ownership.

Fix: explicit ownership in every prompt. Coordinate notes name: (a) the other agents running in parallel, (b) the files those agents own (and that the recipient must not touch), (c) any append-only-shared files.

Pre-flight check (Appendix G): for every file in the project, exactly one role should be able to point at it as theirs. Files that don't match any role's ownership are unowned and get neglected; files that match multiple roles' ownership become conflicts.

Token-budget exhaustion mid-task

Symptom: an agent finishes mid-task with no warning, leaving the work incomplete.

Root cause: the prompt asks for more than the agent's budget can deliver.

Fix: smaller prompts (split the work into more, smaller priorities) and the "stop at boundary" instruction. If a single task is larger than the agent can complete, that's a sign the task should be split into two or three priorities.

A useful heuristic: if the task is "implement X" and X has more than ~300 lines of expected code, split X into 3-4 sub-tasks.

Drift between PRD and implementation

Symptom: the staff engineer ships something that diverges from the PRD's spec.

Root cause: the PRD was wrong, ambiguous, or unactionable; or the staff engineer made a local optimization the PRD didn't anticipate.

Fix: the staff engineer prompt should explicitly license divergence with documentation:

If you find PRD 02 ambiguous or wrong, deviate as needed and document the deviation in your issue file as a **medium**-severity issue.

This both gets you better implementations (the agent isn't forced into a bad spec) and gets you documented divergence (the integrator can decide whether to update the PRD or revert the deviation).

Integrator burnout

Symptom: integration commits start sliding. Resolved files become terse. Issue triage gets cursory.

Root cause: integration is real work, and you're trying to fit it around your day job.

Fix: budget honestly. The case study's integrator estimates 8-10 hours per sprint of integrator time, and protects that time. If you're allocated less, run shorter sprints (one-week sprints with smaller scopes), not corners-cut two-week sprints.

Agents commit anyway despite the "do not commit" instruction

Symptom: `git log` shows commits authored by an agent.

Root cause: the agent platform's tooling allows commits, and the prompt's "do not commit" instruction wasn't strong enough.

Fix: in addition to the prompt instruction, configure the agent's environment to prohibit `git commit` (some platforms support this). If you can't, audit `git log` post-sprint and revert any agent commits, replacing them with your own integration commit.

Tech writer manufactures issues

Symptom: the tech writer files 20 nitpicks per sprint, mostly stylistic preferences.

Root cause: the tech writer prompt didn't constrain.

Fix: explicit instruction. From the case study:

Don't be picky for stylistic preferences — flag genuinely unclear bits only. Don't manufacture issues; clean reviews are valid.

If a tech writer files >5 issues per sprint and most are low-severity, dial back the prompt next sprint.

Chapter 15. Patterns and antipatterns

A condensed reference of what to do and what not to do, with a one-line rationale each.

DO

- **Commit prompts before dispatching.** Audit trail; reproducibility.
- **Make tasks priority-ordered with “stop at boundary” license.** Agent gracefully handles budget exhaustion.
- **Specify file paths in every task.** Agent can't write to the right place if you don't say where.
- **Inline API signatures when you know them.** Removes ambiguity.
- **Use append-only-shared semantics for Makefile, CONTRIBUTING.md, go.mod, etc.** Avoids merge conflicts.
- **Run tech writer in Wave 2, after Wave 1 integrates.** Reviews the integrated tree.
- **Cap the agent's final report at ~200 words.** Forces useful summaries.
- **Reuse 80% of last sprint's prompt scaffolding.** Compounds quality across sprints.
- **Tag releases at multi-sprint boundaries when the product is materially better.** Single-sprint releases are usually a sign of over-tagging.
- **Calibrate severity definitions every 3-4 sprints.** Severity drift is real.

DON'T

- **Don't let agents commit.** Only the integrator commits; one commit per sprint integration.

- **Don't dispatch tech writer in Wave 1.** They have nothing to review yet.
 - **Don't write PRDs as marketing.** Marketing copy doesn't translate to code.
 - **Don't skip the "Out of scope" section of a PRD.** Unbounded scope = scope creep.
 - **Don't half-finish a task.** "Stop at boundary, file issue" is the rule.
 - **Don't share files without ownership.** Unowned files rot; shared files conflict.
 - **Don't run more than four roles per sprint.** The integration cost grows super-linearly.
 - **Don't rely on agent memory across sprints.** Each sprint's prompts are self-contained.
 - **Don't ship without the resolved file.** "We didn't have any issues" is not a valid resolved file.
 - **Don't conflate sprint and release.** Some sprints don't tag; that's normal.
-

Part V — Shipping

Chapter 16. From sprint to release tag

A sprint either ships a release or it doesn't. This chapter formalizes when each is the right call.

Release-gate criteria, in order

To tag a release, all of the following must be true:

1. **The relevant sprint's "Gate to next sprint" criteria, all met.** PLAN.md is your contract; if a criterion isn't met, the gap goes into a blocker issue and the next sprint inherits it.
2. **All previous releases' criteria still hold.** No regressions. CI must be green; manual smoke tests must pass.
3. **The release-specific definition of done is met.** PLAN.md's "Definition of done — per release" section is a checklist. Every box checked, or every unchecked box has a documented reason.
4. **No blocker or high-severity issues open** from the current or any prior sprint without an explicit accept-and-defer rationale.

If any of these is missing, you do not tag. You either fix the gap (preferred) or document the gap and slip the tag (acceptable).

Definition of done — per release

Each release has its own DOD. The case study's are illustrative:

v0.8 (M2) - Sprint 2 complete - `roksbnctl k get/apply/logs/exec/port-forward` covers BNK-relevant operations - Doctor downgrades `kubectl/oc` to informational - Byte-equivalence test green for representative resources - Book at 13 chapters covering Concepts + Getting Started + Cluster Lifecycle + early Operations

The DOD is user-visible: every bullet is something a user can verify by running the binary and reading the docs. “Tests pass” is good; “tests pass and the binary's `--help` shows the new commands” is better.

Deferred work, captured

Every release will have something you wanted that didn't ship. The discipline is to capture it explicitly. The case study has a `## What's deliberately deferred to post-v1.0` section in `PLAN.md` that lists, by category (code / book / methodology), what's been intentionally deferred.

When deferring:

- Name what's deferred specifically. “OpenShift CRDs in `roksbnctl k get` (Project, Route, etc.)” beats “OpenShift extensions.”
- Name where it's tracked. “Tracked in PRD 02 § Phase 2.1.” or “Sprint 5 polish per `PLAN.md`.”
- Name why. “State-handling work — defer to v1.1.” beats silence.

Stakeholders trust release notes that say “we deferred X to v1.1 because of Y” much more than release notes that omit the deferral entirely. They'll find out anyway; you might as well surface it.

Release artifacts

For a CLI / library / tool product, release artifacts include:

- **Binaries** for each supported platform (Linux x64, macOS x64+ARM, Windows x64).
- **Checksums** (SHA256) for each binary.
- **Release notes** summarizing the user-visible changes since the last tag, with links to the relevant chapters of the book.
- **The book itself**, published to GitHub Pages or equivalent, optionally bundled as a PDF.
- **Optional: a Homebrew formula, a Scoop manifest, an apt repo entry.** Land these once your release cadence is steady (typically post-v1.0).

For agentic projects specifically, also include:

- **The prompt files used in the sprints leading up to this release.** These live in `prompts/sprint<N>/` and are already in git; no extra packaging needed. Just don't .gitignore them.
- **The `PLAN.md` and `PRD` set as-of this tag.** Same: in-git, no extra work.

The reason to include the prompts and the PRDs is reproducibility. A future contributor (human or LLM) should be able to check out the v0.8 tag and re-dispatch Sprint 2 against the v0.7 codebase to validate the methodology.

Chapter 17. Scaling the methodology

Two questions come up once the four-role / single-integrator pattern works for your first project:

1. Can I run two sprints in parallel?
2. Can I run two products in parallel?

Both are possible; both have specific cost.

Two sprints in parallel

Two sprints means eight agents in flight (four per sprint × two sprints) and one integrator.

It works if the two sprints are on **disjoint subtrees** of the codebase. The case study could plausibly run Sprint 5's DNS probe (in `internal/test/dns.go`) in parallel with the start of Sprint 6's e2e expansion (in `scripts/`) — no file overlap, no conceptual coupling.

It does not work if the two sprints touch the same files. Two staff engineers writing different features in `internal/cli/cluster.go` will conflict at integration. The integrator burns more time merging than they save by parallelizing.

Practical guidance: stick to serial sprints until you've shipped your first three releases. After that, identify pairs of upcoming sprints that are file-disjoint, and pilot a parallel-sprint cycle. If integration takes <50% extra time, the pattern works for you.

Two products in parallel

Two products means two integrators (you can't be two people; agentic methodology doesn't change that). The roles, prompts, and tooling can all be shared, but the integrator role is single-threaded per product.

Practical guidance: if you want to ship two products, hire (or partner with) a second integrator. Do not try to run two products as a single integrator; the context-switching cost is enormous.

What you can share across products:

- **Prompt scaffolds.** Once you have a working Sprint 2 staff engineer prompt for product A, product B's analogous prompt is mostly cut-and-paste with project-specific details swapped.
- **Issue file format.** Identical across products.
- **PLAN.md structure.** Identical across products.
- **Sprintwatch (or your equivalent dashboard).** Reads the conventional file structure; if it works for product A, it works for product B.
- **Severity calibration.** Once calibrated, applies across products with minor tuning.

The methodology is product-independent. The integrator is not.

Compliance / regulated environments

Some products have to ship through SOC 2, HIPAA, FedRAMP, or other compliance regimes. The methodology adapts:

- **Add a “compliance” role to Wave 2** alongside (or instead of) the tech writer. The compliance agent reads the integrated tree and files issues against the compliance checklist (audit log requirements, data handling, access controls).
- **Sprint 0's CI matrix expands** to include compliance scans (license auditing, SBOM generation, vulnerability scanning).
- **The PRD has a “Compliance considerations” section** describing what the feature must satisfy.
- **The release-gate adds compliance gates** to PLAN.md's “Definition of done — per release.”

In practice, compliance overhead adds 20-30% to integrator time and ~10% to agent runtime. If your project is in a regulated space, plan for it from Sprint 0; retrofitting compliance after the fact is much more expensive than baking it in.

Appendices

Appendix A. Real prompt templates (verbatim from roksbnkctl)

The following four prompts are reproduced from the case study's prompts/sprint2/ directory. They are presented here exactly as dispatched, with one cosmetic redaction: the absolute project paths in the prompts (e.g., /mnt/d/project/roksbnkctl/) are templated as `${PROJECT_ROOT}` for portability.

A.1 Architect prompt template

You are the architect agent for Sprint N of the `${PROJECT_NAME}` project. Your scope is **book chapter authoring** for the M chapters that need to land for the `v${RELEASE_VERSION}` release. No infrastructure work this sprint – all infra (book.yml, mdBook config, GitHub Pages publish path) is established from prior sprints.

Project location: ``${PROJECT_ROOT}``. The book is `_${BOOK_TITLE}_`, served at ``${BOOK_URL}``.

Read first

- ``docs/prd/${SPRINT_PRD}.md`` – the Sprint N PRD that the staff agent is implementing. Chapter X is the user-facing surface for that work.
- ``docs/PLAN.md`` Sprint N section, especially "Documentation deliverables" – confirms which chapters land this sprint.
- ``book/src/SUMMARY.md`` – existing TOC; do not change ordering or filenames.
- The existing chapter stubs at ``book/src/<chapter-files>`` – placeholder stubs you'll replace with real content.
- Sprint N-1's chapters at ``book/src/<reference-chapters>`` – reference for tone, structure, code-block style.
- ``prompts/sprint<N-1>/architect.md`` – Sprint N-1's architect prompt as a template.

Coordinate with parallel agents

A staff-engineer agent is implementing PRD `${SPRINT_PRD}` across `<staff-files>`. A validator agent is adding fake-clientset unit tests under `<validator-files>`, golden-file byte-equivalence tests, editing CI configs, and patching the e2e script.

****Do not touch their files.**** Your scope is `book/src/<chapter-files>` only.

Tasks

For each chapter below, replace the stub content with real prose. Aim for 150-300 lines per chapter, code-block-heavy. Use relative markdown links for in-book cross-references and GitHub-canonical URLs for PRD links.

Chapter X – `book/src/<filename>` – "<chapter title>"

[Concrete sub-section list with what to cover.]

[Repeat per chapter.]

Style guidance

- Lower-case prose; sentence-case section headers
- Code blocks for any command; inline code for filenames and identifiers
- Cross-reference other chapters with relative links
- Short paragraphs; one idea per paragraph
- Examples should be runnable as written
- When citing PRDs, link as `[PRD \${SPRINT\_PRD}](\${GITHUB\_URL\_TO\_PRD})` – GitHub canonical URL avoids the published-book 404 issue.

Issue tracking

`\${PROJECT\_ROOT}/issues/issue\_sprint<N>\_architect.md`:

```markdown

# Sprint N – architect issues

## Issue 1: short title

**\*\*Severity\*\*:** low | medium | high | blocker

**\*\*Status\*\*:** open | resolved

**\*\*Description\*\*:** ...

**\*\*Files affected\*\*:** ...

**\*\*Proposed fix\*\*:** ...

If clean, file with *\*No issues filed.\**.

## Verification before reporting done

- All chapter files have replaced their stubs with real content

- `mdbook build book/` succeeds locally if `mdbook` is installed
- Internal links resolve
- No "Coming in Sprint N" placeholder text left in any chapter

## Final report (under 200 words)

- Per-chapter line count
- Whether `mdbook` was available locally and whether the build worked
- Issues filed (counts by severity)
- Anything the integrator should know

Do NOT commit. The integrator commits the aggregated work.

## A.2 Staff engineer prompt template

```text

You are the staff engineer agent for Sprint N of the `${PROJECT_NAME}` project.
Your scope is `**PRD ${SPRINT_PRD}**` – [one-line description].

Project location: ``${PROJECT_ROOT}``. Module ``${MODULE_PATH}``. Min language
version: `${LANG_VERSION}`.

Read first

- ``docs/prd/${SPRINT_PRD}.md`` – your authoritative spec. The "Implementation tasks" section lists the deliverables in priority order.
- ``docs/PLAN.md`` Sprint N section – confirms ordering and verification gates.
- Existing files in ``<package>`` – the relevant existing code, build on it rather than introducing parallel implementations.
- ``prompts/sprint<N-1>/staff.md`` and the resolved files at ``issues/resolved_sprint<N-1>_*.md`` – patterns from prior sprint.

Coordinate with parallel agents

An architect agent is replacing chapter stubs with real prose under ``book/src/``. A validator agent is adding tests at ``<package>/*_test.go``, editing CI configs, patching the e2e script.

****Do not touch their files.**** Specifically: don't write ``*_test.go`` files in ``<package>`` (validator owns those); don't edit ``scripts/e2e-test.sh`` or ``.github/workflows/ci.yml``. You own everything else.

Tasks (priority order – finish from the top down)

If you run out of token budget, stop at the priority boundary you reached and file an issue describing what's deferred. Don't half-finish a task.

```
### Priority 1 – <task name> (`<file paths>`)
```

```
[Concrete description, with API signatures inline if known.]
```

```
```<lang>
```

```
[Function signatures or interface definition]
```

```
[Repeat per priority.]
```

## Verification before reporting done

- <build command> clean
- <test command> clean
- <vet command> clean
- <fmt command> clean
- Binary smoke test: <binary> --help shows the new commands
- [Sprint-specific checks]

## Issue tracking

\${PROJECT\_ROOT}/issues/issue\_sprint<N>\_staff.md. Same format as Sprint N-1. If a priority item is deferred, file an issue documenting what's missing and why. Don't half-finish.

## Final report (under 200 words)

- Files created (count + key paths)
- Files edited
- Build / test / vet / fmt status
- Which priority items completed; which (if any) deferred
- Issues filed
- Anything the integrator should know (especially regarding new dependencies)

Do NOT commit. The integrator commits the aggregated work.

```
A.3 Validator prompt template
```

```
```text
```

You are the validator agent for Sprint N of the `${PROJECT_NAME}` project. Your scope is **unit tests + integration + CI + security review** for the work the staff agent is implementing per PRD `${SPRINT_PRD}`.

Project location: ``${PROJECT_ROOT}``. Module ``${MODULE_PATH}``. Min language version: `${LANG_VERSION}`.

Read first

- ``docs/prd/${SPRINT_PRD}.md`` – the spec staff is implementing; pay special attention to the "Acceptance criteria" section
- ``docs/PLAN.md`` Sprint N "Test deliverables" – your acceptance criteria
- Existing tests under ``<package>/`` – reference for fake-clientset patterns
- ``scripts/e2e-test.sh`` – existing E2E driver
- ``prompts/sprint<N-1>/validator.md`` – Sprint N-1's validator prompt as template

Coordinate with parallel agents

An architect agent is replacing chapter stubs. A staff-engineer agent is implementing PRD `${SPRINT_PRD}` in ``<package>/`` (production code). **Do not touch their files.** You own ``<package>/*_test.go``, ``.github/workflows/``, ``scripts/e2e-test.sh``, ``docs/E2E_TEST.md``.

Specifically: write ``*_test.go`` files in ``<package>/`` for the new packages staff creates, but never the production files themselves. Use the same package name to access unexported symbols.

Tasks

1. Unit tests (``<package>/*_test.go``)

[Per-file test plan: which files to write, what each tests, coverage target.]

2. Integration / golden-file tests

[Higher-tier tests; build tags; what gates them.]

3. CI workflow updates

[Specific edits to ``.github/workflows/ci.yml``.]

4. E2E patch – ``scripts/e2e-test.sh``

[Specific phase patches.]

5. CONTRIBUTING.md updates

[Append-only sections to add.]

6. Security review

[Specific checks: credential leaks, secrets in argv, container metadata, etc.]

Verification before reporting done

- ``<build command>`` clean
- ``<test command>`` clean (unit suite – your `\_test.go` files green)
- ``<integration command>`` works against the relevant live dependency
- `bash -n scripts/e2e-test.sh` clean
- `DRY\_RUN=1 PHASE\_FROM=<phase> ./scripts/e2e-test.sh` shows the new steps cleanly
- ``<fmt command>`` clean for any file you touch

Issue tracking

`\${PROJECT\_ROOT}/issues/issue\_sprint<N>\_validator.md`. Same format as prior sprint. `Severity: roadmap` for forward-looking observations.

Final report (under 200 words)

- Files created
- Files edited
- Test results (unit + integration if available)
- Issues filed (counts by severity)
- Whether DRY_RUN shows new e2e steps cleanly
- Anything the integrator should know

Do NOT commit. The integrator commits the aggregated work.

A.4 Tech writer prompt template

You are the tech writer agent for Sprint N of the `\${PROJECT\_NAME}` project. Read-only review of all documentation produced this sprint, plus example correctness for the new code.

Project location: `\${PROJECT\_ROOT}`. Your scope is **review + issue filing only** – do not edit any files except
`issues/issue\_sprint<N>\_tech-writer.md`.

Context – what the other agents produced this sprint

- **Architect** replaced chapter stubs with real prose under `book/src/`: chapters X, Y, Z.
- **Staff engineer** implemented PRD `\${SPRINT\_PRD}`: <files> (production), <CLI files> (cobra wiring).
- **Validator** added <test files>, edited <CI configs>, patched <e2e scripts>.

Their issue files are at `issues/issue\_sprint<N>\_<role>.md` with corresponding `resolved\_sprint<N>\_<role>.md`. Read them – your job is to find what they missed from a doc/readability/example-correctness angle.

Tasks

1. New chapter quality

For each new chapter:

- Tone consistency with prior sprints' chapters
- Audience alignment
- Code examples runnable: every ``<command>`` snippet should be a real command. Verify against the staff agent's actual implementation
- Cross-references resolve
- No unfilled placeholders
- Sample output realism

2. Chapter X example correctness – the new <feature> surface

Verify:

- Every command in the chapter actually works against the staff agent's implementation
- Top-level aliases work as documented
- Output format flags match what the staff agent's integration exposes
- Mismatches are filed as **medium** severity issues

3. Cross-document drift check

Spot-check cross-references between the new chapters and:

- `docs/PLAN.md`

- `docs/prd/\${SPRINT\_PRD}.md`
- `book/src/SUMMARY.md`

4. README + CONTRIBUTING updates

Sprint N-1 added a highlight bullet and a section. Sprint N should similarly:

- Add a README highlight for [new feature]
- Add a section to CONTRIBUTING

If those updates are missing, file as **medium** severity.

5. Issue/resolved file format consistency

Verify Sprint N's issue files follow the same format as prior sprints.

Issue file format

```
`${PROJECT_ROOT}/issues/issue_sprint<N>_tech-writer.md`:
```

```
``markdown
```

```
# Sprint N – tech writer issues
```

```
## Issue 1: short title
```

```
**Severity**: low | medium | high
```

```
**Status**: open
```

```
**Description**: what's wrong + where + how a reader would notice
```

```
**Files affected**: paths (with line numbers if useful)
```

```
**Proposed fix**: concrete recommendation
```

If genuinely clean, file with the heading and ***No issues filed.***. Don't manufacture issues; clean reviews are valid.

Verification before reporting done

- All chapter files contain real prose (no "Coming in Sprint N")
- All cross-references in the new chapters resolve to existing files
- All commands in the new chapters appear in the actual binary's help output

Final report (under 200 words)

- Files reviewed (counts)
- Issues filed (counts by severity)

- Top 3 noteworthy observations not filed as issues
- Whether you spotted any drift between PRD and the actual delivered surface

Do NOT edit any files (except your issue file). Do NOT commit anything.

Appendix B. File layout reference

The full file layout of an agentic-developed project, with one-line role per directory.

```

${PROJECT_ROOT}/ |— README.md # User-facing project intro + quick install |—
PRFAQ.md # The press-release / FAQ document (Chapter 1) |— CONTRIBUTING.md #
How humans contribute; agentic-methodology pointers |— LICENSE # MIT or whatever
|— docs/ | |— PRD.md # Optional: top-level requirements document | |— PLAN.md
# The sprint roadmap | |— prd/ | | |— 00-OVERVIEW.md # PRD index — the meta
document | | |— 01-FEATURE-A.md # Per-initiative PRDs | | |— 02-FEATURE-B.md
| | |— ... | | |— E2E_TEST.md # End-to-end testing methodology |— prompts/ |
|— README.md # Methodology playbook (this book in miniature) | |— sprint0/ #
Foundations sprint | | |— architect.md | | |— staff.md | | |— tech-writer.md | |
|— validator.md | |— sprint1/ # ... and so on per sprint | |— sprintN/ |— issues/ |
|— README.md # Issue file format documentation | |— issue_sprint0_architect.md #
Filed by agent during sprint | |— resolved_sprint0_architect.md # Written by integrator
post-sprint | |— ... # 4 issue + 4 resolved files per sprint | |— issue_sprintN_.md |—
book/ # User-facing reference manual (mdBook) | |— book.toml | |— src/ | | |—
SUMMARY.md # The book's table of contents | | |— 01-concepts.md # Per-chapter
content | | |— ... | | |— book/ # Build output (gitignored) |— tools/ # Project-internal
tooling | |— docker/ # Tool images for --backend docker | |— sprintwatch/ # The
progress dashboard (Chapter 12) |— internal/ # Production code (Go convention) | |—
... |— cmd/ # Binary entry points | |— ${PRODUCT}/main.go |— scripts/ # Driver
scripts (e2e, smoke, release) |— .github/ |— workflows/ # CI configuration |— ci.yml
|— book.yml # Publishes the book on push |— release.yml # Tags binaries on tag
push

```

The structure is opinionated. The `prompts/`, `issues/`, and `docs/prd/` directories are **load-bearing** for the methodology; deviations from this layout will require corresponding tweaks to you

Appendix C. Definitions of done, per role

A consolidated checklist of what each role must satisfy before reporting complete.

Architect – definition of done

- [] All chapter files for the sprint have replaced their stubs with real prose (≥ 150 lines per chapter).
- [] No "Coming in Sprint N" placeholder text remains in any chapter.
- [] All in-chapter cross-references (relative markdown links) resolve to existing files.
- [] All PRD links use GitHub-canonical URLs (not relative paths that 404 in the published book).
- [] Every example command in a chapter is verified against the actual implementation (via reading).
- [] ``mdbook build book/`` (or equivalent doc-tool build) succeeds locally if the tooling is available.
- [] ``issues/issue_sprint<N>_architect.md`` is written (with sections for filed issues, or ``*No issues``).
- [] Final report submitted, under 200 words, with required fields.

Staff engineer – definition of done

- [] All priority-1 through priority-N tasks completed, **or** deferred at a clean priority boundary.
- [] No half-finished tasks (a task is either entirely done or entirely deferred).
- [] ``<build command>`` clean.
- [] ``<test command>`` clean (the validator's tests pass against the staff's code).
- [] ``<vet command>`` and ``<fmt command>`` clean.
- [] Binary ``--help`` shows the new commands.
- [] All new exports have at least a one-line doc comment.
- [] Dependencies added to ``go.mod`` (or equivalent) are pinned and documented in the issue file.
- [] ``issues/issue_sprint<N>_staff.md`` is written.
- [] Final report submitted, under 200 words.

Validator – definition of done

- [] Unit tests written for every file the staff engineer created.
- [] Integration tests written for cross-component paths, gated by a build tag.
- [] Golden-file / live tests written for byte-equivalence properties, gated by ``-tags=live``.
- [] CI configuration extended to run the new unit tests.
- [] e2e driver patched per the sprint's "Test deliverables" section of PLAN.md.
- [] Security review per the sprint's PRD: no credentials in argv, container metadata, logs, or process.
- [] CONTRIBUTING.md "Running [new test tier] tests" section appended (if applicable).
- [] ``<build command>`` clean, including with the new tests.
- [] ``<test command>`` clean (unit + integration tiers).
- [] ``bash -n scripts/e2e-test.sh`` clean.
- [] ``issues/issue_sprint<N>_validator.md`` is written.
- [] Final report submitted, under 200 words.

Tech writer – definition of done

- [] Read every chapter the architect produced.
- [] Read every code file the staff engineer touched (skimmed; not line-by-line).

- [] Read every test file the validator wrote (skimmed).
- [] Verified each chapter's example commands against the actual implementation.
- [] Verified all cross-references resolve.
- [] Verified PRD-vs-implementation alignment (drift is filed as medium-severity).
- [] Verified the format of `issue\_\*.md` and `resolved\_\*.md` files matches prior sprints.
- [] `issues/issue\_sprint<N>\_tech-writer.md` is written. If clean, file with `\*No issues filed.\*`
- [] No edits to any file other than the issue file.
- [] No commits.
- [] Final report submitted, under 200 words.

Integrator – definition of done (per sprint)

- [] Wave 1 dispatched in parallel; results aggregated.
- [] All Wave 1 issue files reviewed; each issue triaged (fix / accept / defer).
- [] All verification gates run and passing (build / test / vet / fmt / binary smoke).
- [] `resolved\_sprint<N>\_<role>.md` written for each Wave 1 role.
- [] Wave 1 integration committed in a single commit, with a descriptive message naming each role's
- [] Wave 2 (tech writer) dispatched.
- [] Wave 2 issues triaged.
- [] `resolved\_sprint<N>\_tech-writer.md` written.
- [] Wave 2 integration committed.
- [] PLAN.md gate criteria met for this sprint, *or* gap captured in `issue\_sprint<N>\_blockers.md`
- [] If sprint gates a release: tag created, release artifacts published, release notes written.
- [] Stakeholder update sent (per Chapter 12).

Appendix D. PRFAQ template

Save this as `PRFAQ.md` at the project root, then fill in.

```markdown

# PRFAQ – \${PRODUCT\_NAME}

\*Draft, internal only. Not for distribution.\*

## Press release

### \${PRODUCT\_NAME} \${RELEASE\_VERSION}: \${ONE\_LINE\_HEADLINE}

\*\${SUB\_HEADLINE – who is this for, what does it do they couldn't before, in one sentence}\*

**\*\*The problem.\*\*** `${THREE_TO_FIVE_SENTENCES}` describing the user's pain. What were they trying to do? What was in their way? Why is now the right time?

**\*\*The solution.\*\*** `${THREE_TO_FIVE_SENTENCES}` describing the product. What does it do? How is it shaped? How does it solve the problem?

**\*Example.\*** `${A_CONCRETE_EXAMPLE}` of use – actual command lines, actual screen output. If you cannot write this, you do not yet know what the product is.

```
```bash
$ ${COMMAND}
${ACTUAL_OUTPUT}
```

FAQ

For users

How do I install it? `${ANSWER}`

What does it cost? `${ANSWER}`

What's the closest existing thing I'd compare it to? `${ANSWER}`

What if I want to use `${EXISTING_THING}` alongside it? `${ANSWER}`

Where do I get help? `${ANSWER}`

For engineering

What does this require us to build? `${HIGH-LEVEL_OUTLINE}` — initiatives, not features. Reference the PRD index once it exists.}

What's the riskiest dependency? `${SPECIFIC_RISK + MITIGATION}`

What's the smallest version we could ship? `${MINIMAL_FEATURE_SET}`

What's the test plan? `${HIGH-LEVEL_APPROACH}` — unit + integration + e2e}

For executives

What does success look like? `${MEASURABLE}` — “fresh dev box can complete the full lifecycle without installing other tools” beats “users are happy”}

What are we not doing? `${EXPLICIT_NON-GOALS}` — aim for 4-8 bullets}

How long? `${CALENDAR_ESTIMATE_WITH_RANGE}` — “12-16 weeks” beats “Q4”}

What are the open questions? `${LIST}` — each with a default answer the team will use unless overridden}

```
---

# Appendix E. PRD template

Save this as `docs/prd/${NN}-${INITIATIVE_NAME}.md`. Fill in.

```markdown
PRD ${NN} – ${INITIATIVE_TITLE}

Why

${ONE-OR-TWO PARAGRAPHS of motivation. Why this work? Why now? What's the
user pain? Keep this short – agents do not act on motivation; they act on
the rest of the document.}

Goal

${ONE PARAGRAPH stating the desired outcome in measurable terms. "Drop the
kubectl install requirement for the happy path" beats "Improve the UX."}

In scope

- ${BULLETED LIST of capabilities being added – each bullet a sentence. Aim
 for 5-15 bullets.}
- ${...}

Out of scope

- ${EQUALLY IMPORTANT bulleted list of capabilities NOT being added, with
 one-line rationale each. This is your guardrail against agent scope
 creep.}
- ${...}

Implementation tasks

| Order | Item | Files |
|---|---|---|
| 1 | ${SPECIFIC TASK with target file path} | new / edit |
| 2 | ${...} | new / edit |
| ... | ... | ... |

${INLINE API SIGNATURES when known:}
```

```
```${LANG}
// Public API surface this PRD adds:
${SIGNATURES}
```

Acceptance criteria

- \${BULLETED LIST of testable conditions for “done.” These become the validator’s test list.}
- \${...}

Dependencies

- \${OTHER PRDS that must land first}
- \${EXTERNAL LIBRARIES added}
- \${NEW go.mod / package.json LINES}

Risks

Risk	Severity	Mitigation
\${WHAT COULD GO WRONG}	low / medium / high	\${HOW WE’LL HANDLE IT}

Open questions

- \${EXPLICIT QUESTION + PROPOSED DEFAULT — agent picks default unless told otherwise; integrator tracks for follow-up}

Decided (binding for downstream PRDs)

Decision	Rationale
\${LOCKED-IN DESIGN CHOICE}	\${WHY}

Appendix F. Sprint plan template

A sprint plan is a section of `docs/PLAN.md`. Add one section per sprint as you go.

```

```markdown
Sprint ${N} – ${ONE_LINE_TITLE} (${WEEK_RANGE})

Goal

${ONE_SENTENCE: shipping this sprint means...}

Code deliverables

| Order | Item | Files |
| --- | --- | --- |
| 1 | ${TASK} | ${FILES} |

Test deliverables

- **Unit tests** (${PACKAGE}): ${COVERAGE}
- **Integration tests** (${TIER}): ${COVERAGE}
- **E2E patch**: ${WHAT}

Documentation deliverables

- **Chapter ${X}: ${TITLE}** (`book/src/${FILENAME}`) – ${SCOPE}

Gate to Sprint ${N+1}

- ${TESTABLE_CONDITION 1}
- ${TESTABLE_CONDITION 2}
- ${TESTABLE_CONDITION ${K}}

Risks

- ${RISK} – Mitigation: ${MITIGATION}

```

---

## Appendix G. Integrator's pre-flight checklist

Before drafting Sprint N's prompts, walk this list. Each item should answer "yes."

- ☐ Sprint N's PRD has been read end-to-end (not just skimmed).
- ☐ Sprint N's docs/PLAN.md section has been read, including the testing-pyramid additions table.

- ☐ All open issues from sprints  $< N$  have been triaged: rolled into Sprint N's task list, deferred to a later sprint with a note, or closed as won't-fix.
  - ☐ The four prompts are mutually exclusive (no overlapping deliverables) and collectively exhaustive (no scope gaps vs. PLAN.md Sprint N).
  - ☐ Coordination notes in each prompt name the other three agents and their owned files.
  - ☐ Each prompt's "verification before reporting done" block includes the relevant sprint-specific checks.
  - ☐ Tech-writer prompt has been adjusted from the previous sprint's template to call out anything new this sprint.
  - ☐ Each prompt's "Read first" section names  $\geq 4$  files, including the PRD, the PLAN.md section, the previous sprint's analogous prompt, and any relevant existing code.
  - ☐ Each task in each prompt has a target file path.
  - ☐ Each staff-engineer task has either an inline API signature or a reference to where the signature can be read.
  - ☐ No file is owned by two roles. No file in the project is unowned by any role for the duration of the sprint.
  - ☐ The four prompts have been committed in their own commit before any agent is dispatched.
- 

## References

### Primary sources (the case study)

- John Gruber. roksbnctl: a CLI for deploying BIG-IP Next for Kubernetes on IBM Cloud OpenShift. GitHub repository. <https://github.com/jgruberf5/roksbnctl>.
  - The prompts/, issues/, docs/, and tools/sprintwatch/ directories contain the verbatim artifacts cited throughout this book.

### Methodology references

- Amazon's "Working Backwards" framework — the canonical PRFAQ origin. Bryar, Colin & Carr, Bill. Working Backwards: Insights, Stories, and Secrets from Inside Amazon (St. Martin's Press, 2021).
- The Mythical Man-Month — Brooks, Frederick P. — for sprint sizing and the "second-system effect."
- Site Reliability Engineering (Google) — for severity calibration and blameless post-incident review patterns adapted here for the integrator's resolved-file pattern.

## Tooling references

- mdBook — <https://rust-lang.github.io/mdBook/> — the book-authoring tool used by the case study.
- Bubble Tea — <https://github.com/charmbracelet/bubbletea> — the Go TUI framework used by sprintwatch.
- Lipgloss — <https://github.com/charmbracelet/lipgloss> — the Go terminal styling library used by sprintwatch.
- testcontainers-go — <https://golang.testcontainers.org/> — the Go integration-testing pattern referenced in the case study's validator prompts.

## LLM platform references

- Claude — Anthropic's LLM, used as the agent platform for the case study.
  - Claude Code — Anthropic's terminal-based agent harness.
- 

## Index

Page numbers omitted in this single-file edition; entries point at chapters and sections.

- **Acceptance criteria** — Chapter 3 (PRD format), Appendix C (per-role)
- **Agent platforms** — Preface, References
- **API drift** — Chapter 13
- **Append-only-shared files** — Chapter 13, Chapter 15
- **Architect role** — Chapter 5, Chapter 8 (prompt), Appendix A.1, Appendix C
- **Burnout (integrator)** — Chapter 14
- **Burn-down chart** — Chapter 12
- **Calibration (severity)** — Chapter 11, Chapter 15
- **Compliance** — Chapter 17
- **Coordination notes (in prompts)** — Chapter 8, Appendix G
- **Definition of done** — Chapter 16, Appendix C
- **Deferred work** — Chapter 11, Chapter 16
- **Dispatch (Wave 1, Wave 2)** — Chapter 5, Chapter 6, Chapter 10
- **Documentation infrastructure** — Chapter 9
- **End-of-sprint commit** — Chapter 6, Chapter 10
- **Failure modes** — Chapter 14
- **File layout** — Appendix B
- **Half-finished implementations** — Chapter 8, Chapter 14
- **Hallucinated APIs** — Chapter 14
- **Initiative (definition)** — Chapter 2

- **Integrator role** — Chapter 6
- **Issue file format** — Chapter 11
- **Issue ledger** — Chapter 11
- **mdBook** — Chapter 9, References
- **Out of scope (PRD section)** — Chapter 3, Chapter 15
- **Parallel sprints** — Chapter 17
- **PLAN.md** — Chapter 4, Chapter 7, Appendix F
- **Pre-flight checklist** — Appendix G
- **PRD index** — Chapter 4
- **PRD format** — Chapter 3, Appendix E
- **PRFAQ format** — Chapter 1, Appendix D
- **Priority ordering (in tasks)** — Chapter 8, Chapter 14
- **Prompt commit (before dispatch)** — Chapter 6, Chapter 15
- **Prompt structure** — Chapter 8
- **Read-first section** — Chapter 8, Chapter 14
- **Release tags** — Chapter 7, Chapter 16
- **Resolved file format** — Chapter 11
- **Roadmap severity** — Chapter 8 (validator), Chapter 11
- **Roles (architect/staff/validator/tech-writer/integrator)** — Chapter 5, Chapter 6
- **Severity values** — Chapter 11
- **Sprint 0** — Chapter 7, Chapter 9
- **Sprint anatomy** — Chapter 7
- **Sprintwatch** — Chapter 12
- **Stakeholder reporting** — Chapter 12
- **Staff engineer role** — Chapter 5, Chapter 8 (prompt), Appendix A.2, Appendix C
- **Stop at boundary (priority)** — Chapter 8, Chapter 14
- **Tech writer role** — Chapter 5, Chapter 8 (prompt), Appendix A.4, Appendix C
- **Templates (PRFAQ / PRD / sprint)** — Appendices D, E, F
- **Validator role** — Chapter 5, Chapter 8 (prompt), Appendix A.3, Appendix C
- **Verification before reporting done** — Chapter 8, Appendix C
- **Wave 1 / Wave 2** — Chapter 5, Chapter 6, Chapter 10
- **“Working Backwards”** — Chapter 1, References

---

End of book.